



MUXP Manual

24.01.2021 for MUXP version 0.4.0

schmax

nofaceinbook

⚠ This manual is intended to be read by people who want to really understand MUXP and all it's options. If you just want to simply update your mesh the README file should be sufficient.

NEW New sections or changes from older manual version for MUXP 0.3.2 are in blue font.

Content

Content	1
YAMET - Yet Another Mesh Editing Tool	3
How Does MUXP Work	3
Installation	5
Windows - Easy Way	5
Windows - Flexible Way	5
All Operating Systems	5
Updating MUXP	6
Running MUXP	6
Drag & Drop (Windows Executable)	6
Using MUXP GUI	7
Issues when Updating Mesh	8
Airport Area is Flattened	8
Mesh Updated is Currently Not Active	9
Select Scenery to Update	10
No Matching Scenery	10
Updating Already Updated Mesh Files	11
Errors and Warnings	12
MUXP Configurations	13
Export to kml to see Changes of MUXP in Google Earth	15
Creating Your First MUXP File	16
Automatic creation from apt.dat	17
Check and Update MUXP in Google Earth	21
Use WED	30
Share MUXP file with others	34
MUXP File Structure	34
Example File	34
Must Base Values	37
Optional Base Values	37
MUXP Commands	38
cut_polygon:	38
cut_ramp:	38

cut_flat_terrain_in_mesh:	38
cut_spline_segment :	39
cut_spline_poly:	39
cut_strip:	39
cut_path:	40
limit_edges :	40
update_raster_elevation :	40
update_raster4spline_segment:	41
update_elevation_in_poly:	41
extract_mesh_to_file:	41
insert_mesh_from_file:	43
exit_without_update:	45
update_network_levels:	45
unflatten_default_apt:	46
calculate_vertex_normals:	46
Undo Mesh Updates and/or Uninstall MUXP	49
Basic Knowledge on DSF Files	50
Outlook & Feedback	50
Credits	50

YAMET - Yet Another Mesh Editing Tool

Probably you are thinking not again another mesh editing tool, which is complicated and does not really help to solve the problem for changing the mesh in X-Plane. However, this tool is trying to overcome issues of other mesh tools. Highlights are:

- Option to change the mesh and thus elevation for the area e.g. for your airport and share these changes with others without the need to share complete dsf files which are large, might have copyright issues and change only your area (e.g. not if there are changes required for another airport).
- Better flatten airports only where needed (not a huge area as done by the X-Plane flatten flag)
- Generate runway profiles exactly as they are
- You can also change the elevation of raster squares
- See how the mesh really looks like via .kml files that can e.g. be visualized with Google Earth
- Option to define muxp changes in Google Earth or also in WED
- Export/import the physical base mesh to/from .obj file e.g. for performing further changes in blender
- Also update the elevation layers for networks/roads to e.g. generate bridges.

Important is that MUXP is not meant to be the final solution to solve the missing mesh editing functionality in X-Plane. It should just help as an intermediate fix until a real solution is provided by LR.

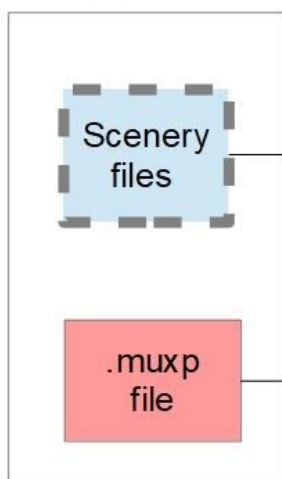
How Does MUXP Work

Muxp adapts the dsf-files of X-Plane that do include the mesh information (not the dsf files including overlays or e.g. airport buildings). For the adoption the scenery developer creates a muxp file (yaml text file) including definitions for the mesh like flattening of a certain area. This muxp file is submitted e.g. with other scenery files (e.g. new airport) to the user. The user installs scenery as normal. The muxpf file is then processed by this muxp tool. The tool is searching automatically for installed mesh files (such as Ortho, HD Mesh or even already adapted dsf files) and the user can decide which one to be updated. Processed default dsf files are stored in the muxp updated mesh folder within Custom Scenery. Adapted custom meshes will stay in their Custom Scenery subfolder and the original custom dsf is saved as a file with ending '.muxp.original'. When X-Plane starts the adapted mesh will be shown (in case there is no other custom mesh with higher priority in

scenery_packs.ini file. Refer to available documents and YouTube tutorials how to organize your *scenery_packs.ini* file in the right order, in case you have issues.

Scenery Developer

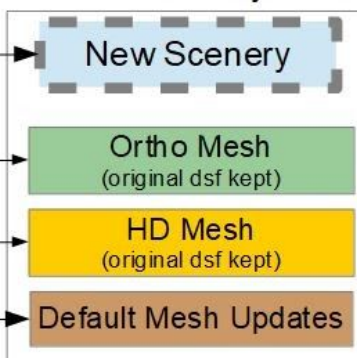
Scenery Download Package



User

X-Plane Folder

Custom Scenery Folder



Global Scenery Folder



muxp Tool

Original custom dsf are backed up to ...dsf.muxp.original
Previous mesh update is stored in ...dsf.muxp.backup

Installation

⚠ MUXP is still in development and has therefore still errors. Use with care at your own risk.

□ MUXP can be downloaded either at X-Plane forum:

<https://forums.x-plane.org/index.php?/files/file/67230-mesh-updater-x-plane-muxp/>

or github: <https://github.com/nofaceinbook/muxp/releases/latest>

Windows - Easy Way

For windows you could just use the executable file. Unzip the folder “zzzz_MUXP_Mesh_Updates” to your “Custom Scenery” folder in X-Plane. Done.

Windows - Flexible Way

You could install the muxp.exe file to any other location on your PC. But when you do this outside the Custom Scenery folder of X-Plane you have to specify your X-Plane root folder and the folder where changed default mesh files will be stored in Custom Scenery. MUXP will ask you for these two folders when you run it the first time and save this information in the file “muxp.config” for future runs.

All Operating Systems

MUXP is written in Python. For the moment there are no executables for MacOS or Linux. For these OS (and if you also like Windows) you need to run Muxp with Python Version 3.8 (64-Bit). You also need to install Python modules listed in “[requirements.txt](#)”. If you are not sure how to install such modules and how to run Python scripts please search the Internet.

The program you need to run at the end is “python mxup.py”.

If you install all the MUXP Python scripts in a folder like “zzzz_MUXP_Mesh_Updates” inside your “Custom Scenery” then Muxp can directly use this folder for updated default mesh files and knows where X-Plane is located on your drive. But you could install it to any other location and give this information on the first start of MUXP.

Updating MUXP

MUXP is still under development, so check for updates at download locations.

In case of executable you just need to replace muxp.exe and in case of python scripts, all python scripts (ending .py).

Running MUXP

In order to update your mesh with MUXP you need a MUXP file including all information on the update that should be performed. Usually these files have the ending ".muxp".

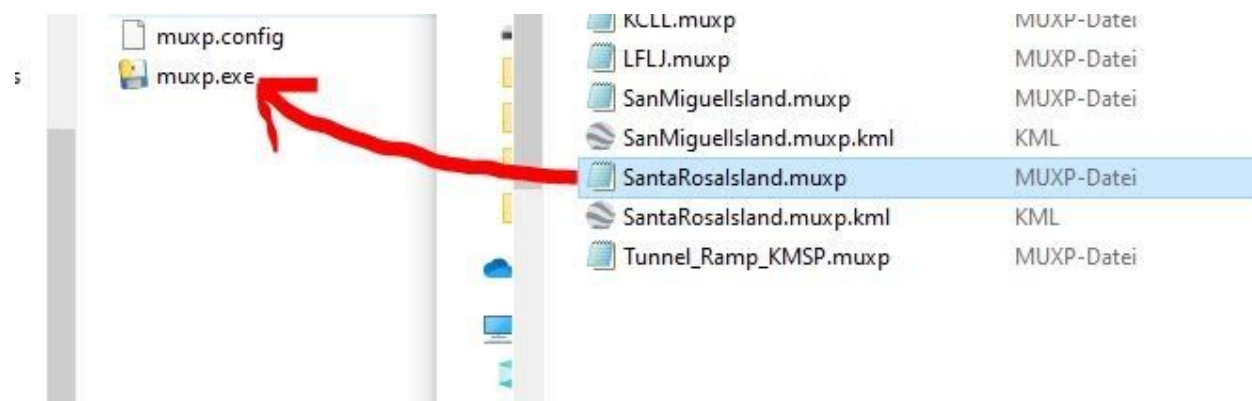
i The filename does not need to end with ".muxp". It could have any name. MUXP can also process ".kml" files or "wed.xml" files as long as these files have the MUXP information injected.

□ To get started, you can download some MUXP files here:

<https://github.com/nofaceinbook/muxp/tree/master/muxpfiles>

Drag & Drop (Windows Executable)

The easiest way is just to drag your file within Explorer and drop onto muxp.exe using your mouse. MUXP will then automatically process this file.



You can also drag and drop folders with MUXP files and all files in the folder will be processed one after the other.

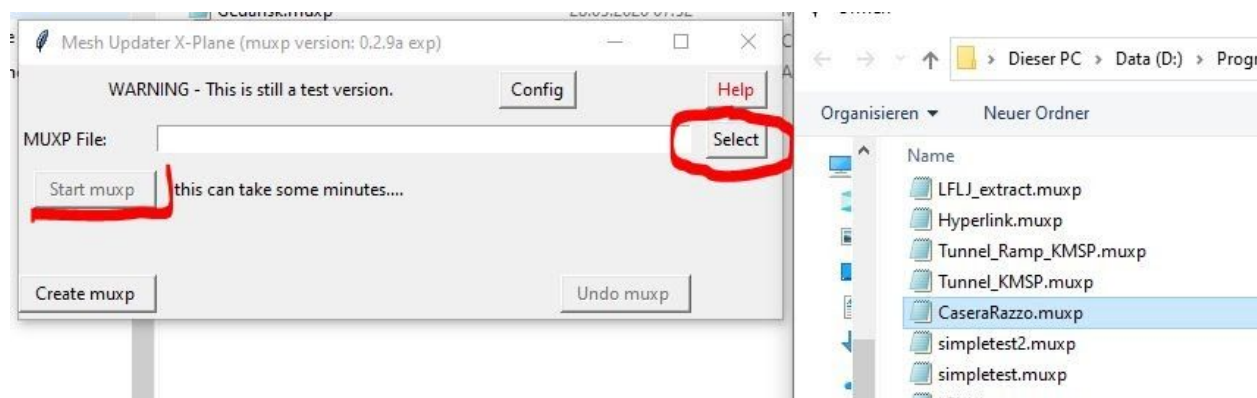
⚠ Only include MUXP files inside such a directory in order to avoid errors.

⚠ Don't put too many files in one folder as this could take a long time.

Using MUXP GUI

What works for all operating systems and what is the default way is to just run muxp (either the executable or by calling "python muxp.py").

A small window opens where you have a button to select a MUXP file via the file browser. Browse to the MUXP file you want to use for updating the mesh, select it and then press "Start muxp" in the main window. Note that depending on the update and the mesh processing could take some minutes.



i Note that the file type can be changed in the file browser in case you want to open not only .muxp files but .muxp.kml, .wed.xml, .yaml or any other file.

After you run MUXP your mesh update is ready to be used like an unusable terrain for a runway becomes now usable and could include other changes like added lakes.



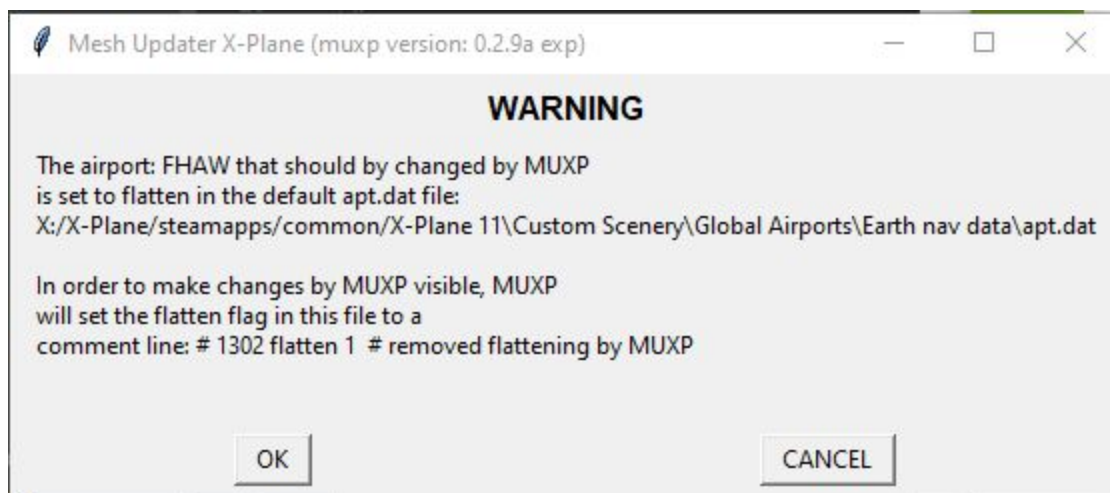
Issues when Updating Mesh

In case you just update your default X-Plane mesh with well tested MUXP files you should not have any issues. During the run MUXP shows in the main window the current processing status like reading dsf-file, processing muxp command or writing dsf-file. When completed the main window shows a success message like: *"Finished Mesh Update xy" and "Scenery Pack z adapted"*.

However, as mesh definitions for X-Plane are quite complex you could encounter special situations that have to be handled. MUXP tries to guide you to these situations via dialog windows and suggests the preferred option. For your interest you see below some of the most common issues you could encounter.

Airport Area is Flattened

As there is no easy way to adapt the mesh X-Plane offers scenery designers the option to just flatten all the area around an airport. This is of course you usually don't want when you want to update the mesh for the airport like creating a profiled runway. Therefore MUXP can update the apt.dat for the global default airports (situated in X-Plane 11/Custom Scenery/Global Airports/Earth nav data). It will just set the flatten command (1302 flatten 1) for the relevant airport to a comment line including the additional comment "removed flattening by MUXP" (so you could search for it and adapt it again your own when needed).



- i** MUXP only removes flatten flags for default airports. It is expected that scenery designers crafting additional airports take care of not including such flattening when scenery should be used with MUXP. However, if you have e.g. an old scenery where this flag is included you could just open the apt.dat for this airport with an editor, search for "1302 flatten 1", just place '#' in front and save the file.

- i In case you still just see flatten area around your airport, please make sure that in global X-Plane settings you have enabled: "runways follow terrain contours".**

Just to be sure, before doing any change, X-Plane also creates a backup copy of your global airports apt.dat file called *apt.dat.beforeMUXP*.

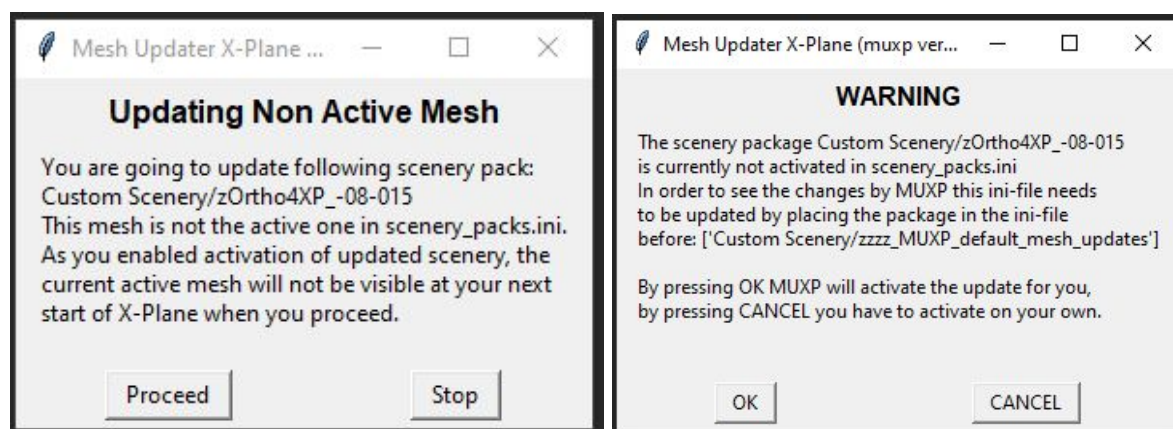
Mesh Updated is Currently Not Active

You could have several meshes installed in X-Plane but with regard to the base mesh there is only just one mesh that will be used (you might have overlay meshes defining e.g. roads or forest, but no further base mesh with elevation changes). The mesh used is the active one, which is the one topmost in your *scenery_packs.ini* file, or the default X-Plane mesh if this file has no entry for that area.

Typical scenario is that you have installed an additional Ortho-Mesh (Ortho-Mesh always has the base mesh included). When you now process a MUXP file that was specifically defined for the default X-Plane mesh it would just update it but you would still see your previous Ortho-Mesh. Therefore you will be notified by MUXP whether you would like to proceed in updating the inactive mesh or stop e.g. to run a MUXP file updating your Ortho-Mesh.

- i MUXP default setting is (could be changed, refer to MUXP Configuration) that if you decide to proceed, MUXP will set the updated mesh to the active one in scenery_packs.ini. But there will be a second window where you have to confirm changing scenery_packs.ini.**
- ⚠ Activation of sceneries changes the order of sceneries in scenery_packs.ini. The scenery that will be activated is placed before the current active pack. As meshes often cover several areas in the world (tiles) this might also have effects for the other areas. So it might sometimes be better to set the right order yourself supported by tools like XOrganizer.**

In case activation of mesh is activated (default setting) you get two notifications. One before the update process starts and one after processing if scenery_packs.ini should be really changed:

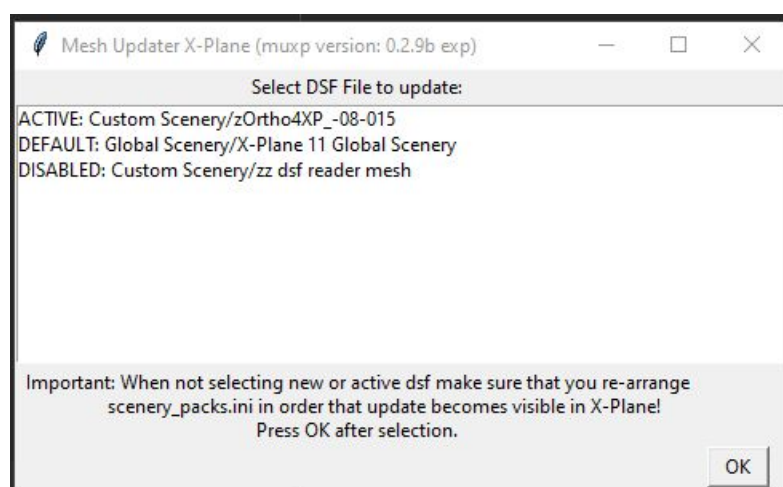


When MUXP updates scenery_packs.ini it also creates backup files *scenery_packs.backupMUXP* (for every change done) and *scenery_packs.beforeMUXP* (the first time a backup is created).

Select Scenery to Update

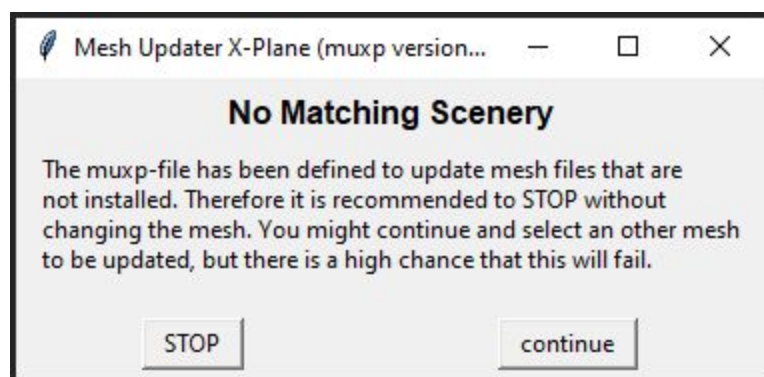
In case the MUXP file does not define (optional key: *source_dsf*) which scenery it should be used for and you have several sceneries installed, then you can decide which scenery pack you would like to update. Usually you would update your active mesh.

But be warned that not each MUXP file works for every mesh. So you might try to run it on another pack if the one before produces errors. Therefore it is better when the creator of a MUXP file is testing for which mesh it works and specify those in the MUXP file.



No Matching Scenery

It might be that the MUXP file is defined for specific meshes that currently are not installed in your X-Plane. In that case you should either install a matching mesh (refer to "source_dsf:" in MUXP file) or use a MUXP file working for your installed mesh.



Updating Already Updated Mesh Files

One main aspect of MUXP is to have several updates in one mesh. So you should not be required to have an own dsf file for every airport that you need to activate depending on the airport you are flying from/to.

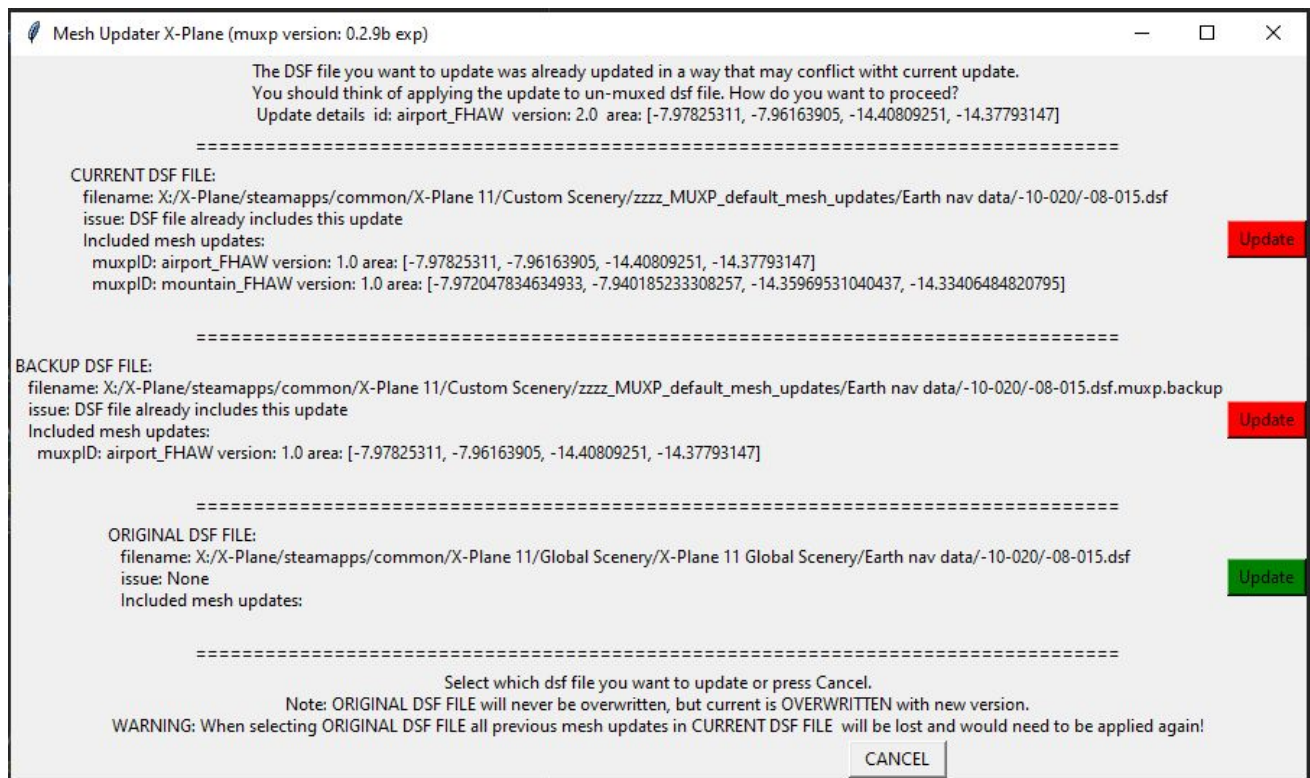
Updating an already modified dsf file is no issue as long as you update another area of the dsf tile. If updating overlapping areas, MUXP will just do it as long as MUXP file has the same id; so you can easily update newer versions of mesh updates. However this will normally not work in case there was another update of the same area in-between. As the MUXP-file usually is defined for the state of the mesh before the update, running the same/similar update again might end up in errors. For that reason MUXP usually uses backup/original dsf files and just applies for them the new update.

In case you have applied several changes for one dsf-file and want to run e.g. a new MUXP file version for the first update then the recommended way is to recreate the updated dsf file again starting from original, apply the updated MUXP file and then run again all other updates to this new dsf file.

i MUXP keeps all applied MUXP files in the folder “Installed MUXP Files” inside your default MESH update folder. So you have always the chance to recreate an updated dsf file again starting from the original file.

It is planned that a future version of MUXP will also handle the case of updating an already created update after other changes have been applied by just automatically applying all MUXP files again. However, for the moment you have to do this by hand.

The following example shows how conflicts with several updates are handled.

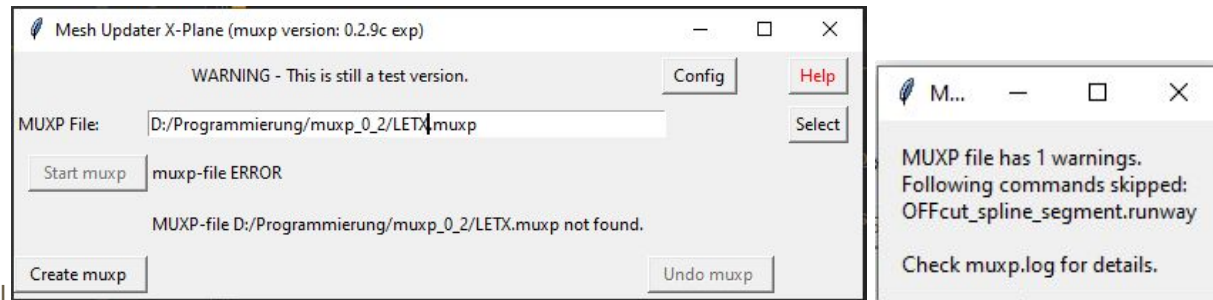


In that example the dsf file you want to update includes already two updates with MUXP file IDs: airport_FHAW, version 1.0 and mountain_FHAW, version 1.0. In case you would like to install a new version of the mountain change, this would not raise any issue. In that case MUXP would directly take the BACKUP DSF file which includes the airport and would just update the new version of the mountain. In our case however we would like to install version 2.0 of the airport. CURRENT and BACKUP DSF files already include version 1.0 of the airport. So installing version 2.0 of the airport on top would probably end up in errors. For that reason you would install the mesh update on the ORIGINAL DSF file (green button). The only problem is then, that the original does not include your change of the mountain. So you have to install this MUXP file again on your newly created dsf file which includes now version 2.0 of the airport. You will find the required file (if you don't have it any more) in the "Installed MUXP Files" folder which in our case has the name: *-08-015 mountain_FHAW 1.0.muxp*.

Errors and Warnings

Typical errors like files not found are directly displayed on the main MUXP window. In case of warnings or errors in the MUXP file itself an additional window shows what is wrong. In case of warnings MUXP has still enough information in order to create the update. In case of errors MUXP will stop.

- i** In case of errors you could also refer to the log-file muxp.log which is stored in the folder where MUXP lives (muxp.exe or muxp.py). This file could be huge so you could search for keywords like error or warning.



As MUXP is still in the development phase, also unforeseen and not handled errors or behaviours could occur. It could also happen that MUXP hangs. If you don't see any new process messages on the MUXP main window during a run then there was probably an error. In that case you could close the window and perhaps try with slightly adapted coordinates of the MUXP file command that did not continue again.

In case you have errors on certain aspects please let me know in order to improve MUXP further. Best way would be to open an issue on github:

<https://github.com/nofaceinbook/muxp/issues>

- i** Most often errors occur in certain edge cases when cutting mesh triangles. Especially in case of parallel lines or very close vertices. So you might check how the mesh and your cutting lines are situated (refer to kml export below) and just slightly change coordinates.

MUXP Configurations

In the configuration Menus of MUXP, which you can reach via the "Config" button from the main menu, you can perform different settings. This is normally only relevant if you have advanced needs, especially if you want to create and test your own MUXP files.

X-Plane base folder: is the path to the X-Plane 11 folder of your X-Plane installation, which needs to be known by MUXP.

Folder to updated DEFAULT dsf: Default X-Plane mesh is left in its original states where it resides in X-Plane. If the default mesh is changed a copy is created to the Custom Scenery folder and adapted there. This is the path to the folder where the updated default mesh is

stored within the X-Plane Custom Scenery folder. The button **Create folder for updated dsf** will create such a folder called “zzzz_MUXP_default_mesh_updates”.

Export to kml: If ticked MUXP creates .kml files before updating the mesh and also after each processed command. The .kml files are stored within your folder where MUXP lives (muxp.exe or muxp.py). They are named with regard to the adapted tile coordinates and the number at the end is increased after each processed command (e.g. -08-015_dsf_0.kml). These .kml files can be viewed with Google Earth and really help when creating your own mesh. Also refer to the section on this below for an example.

Activate Pack: If ticked and you update a non active base mesh MUXP will allow you to automatically set the updated mesh to the active one by adapting scenery_packs.ini accordingly. Refer also to section “Mesh Updated is Currently Not Active” above.

Mesh Updater X-Plane (muxp version: 0.2.9a exp)

--- SETTINGS FOR MUXP ---

X-Plane base folder: X:/X-Plane/steamapps/common/X-Plane 11 Select

Folder to updated DEFAULT dsf: X:/X-Plane/steamapps/common/X-Plane 11/Custom Scenery/zzzz_MUXP_defau Select

Create folder for updated dsf <-- updates scenery_packs.ini

ADVANCED SETTINGS (Defaults are empty fields)

☐ Export to kml ☒ activate Pack <-- updates scenery_packs.ini

Fixed source for dsf (optional): Select

^ Option to use [ACTIVE]. SELECT searches all packs and takes a while! ^

Conflict strategy (optional):

^ Options are: IGNORE, CURRENT, ORIGINAL, BACKUP, CANCEL

SAVE

Fixed source for dsf: If you specify a scenery pack like default X-Plane mesh or a HD Mesh etc. then MUXP will always use this scenery pack for adapting it. This allows you on the one hand free choice and on the other hand it speeds up processing as MUXP has not to identify available meshes itself. You could also set the value to [ACTIVE] which means that always the current active mesh will be updated.

Conflict Strategy: In case you update an already updated area that could end up in conflicts, which are usually solved by decision of the user (refer to section “Updating

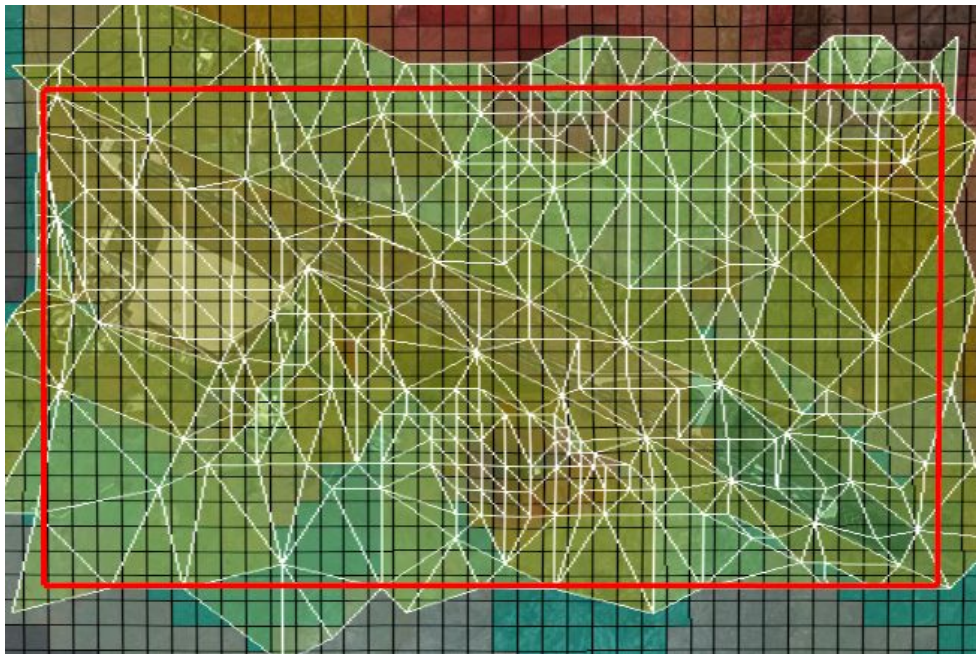
Already Updated Mesh Files” above). In case you want to pre-define the decision, you could do this here. This is especially helpful in case you are testing again and again the same MUXP file. Options are:

- IGNORE: Regardless, update the mesh, even if there is a conflict. This option is not recommended as it usually ends up that you update an already updated mesh which usually ends up in errors.
- CURRENT: Updates current mesh file, which is also not recommended.
- ORIGINAL: This uses the original mesh before any update. This is usually the value you use for testing a new MUXP file again and again.
- BACKUP: This uses the backed up MUXP file from the last change. This option is usually used in case you want to test a MUXP file on an already updated mesh.
- CANCEL: If you select CANCEL that processing is directly aborted in case of a conflict.

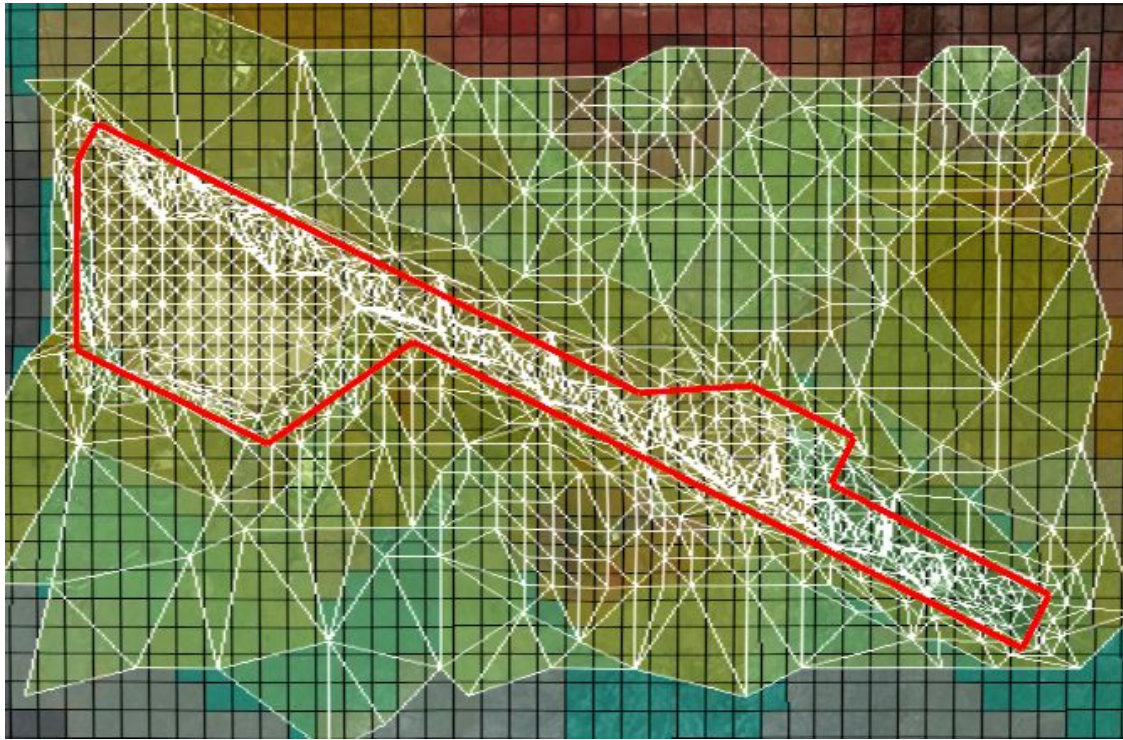
i All these configuration settings are stored in the file muxp.config which has to reside in the same folder as MUXP (muxp.exe or muxp.py).

Export to kml to see Changes of MUXP in Google Earth

This example shows the kml-export before MUXP is adapting the X-Plane default mesh (whit triangles) on airport FHAW within the area of the red rectangle:



After the MUXP file - that is explained within the next section - was processed, the mesh looks like this:



One command “cut_spline_segment” sliced the runway in small segments in order to smoothly have a profiled runway. The command “limit_edges” ensured that all triangles of the mesh within the red polygon (in that case the airport boundary) have no edges longer than 100 meters. This ensures that the triangles reflect more accurately the elevations.

The colored squares in the diagram show the rasters that define in X-Plane default mesh the elevation. Blue/green squares have a low elevation, where brown/red squares have a high elevation.

Creating Your First MUXP File

This chapter gets you started creating your own MUXP files by using airport definition files (apt.dat), Google Earth and WED. Press in MUXP the button “Create muxp” to get a GUI for making use of these options.

The text field on the button of this GUI window can be scrolled and it displays if files have been created or errors that occur.

Automatic creation from apt.dat

In this example we adapt the private Lost Bridge Village airport in Arkansas, in the middle of the US. It is nicely situated on a ridge which ends on one side at Beaver Lake and on the other side a hill (which makes it usually only be used to/from the lake side).

Below you see how “bad” this airport looks like in default X-Plane. As the mesh was not specifically adapted for this airport, the flatten flag was used which creates a huge flat area around the airport and makes the lake running up the hill.

Now we use MUXP to make the mesh look like in reality. Click on the “create muxp” button in the main GUI.

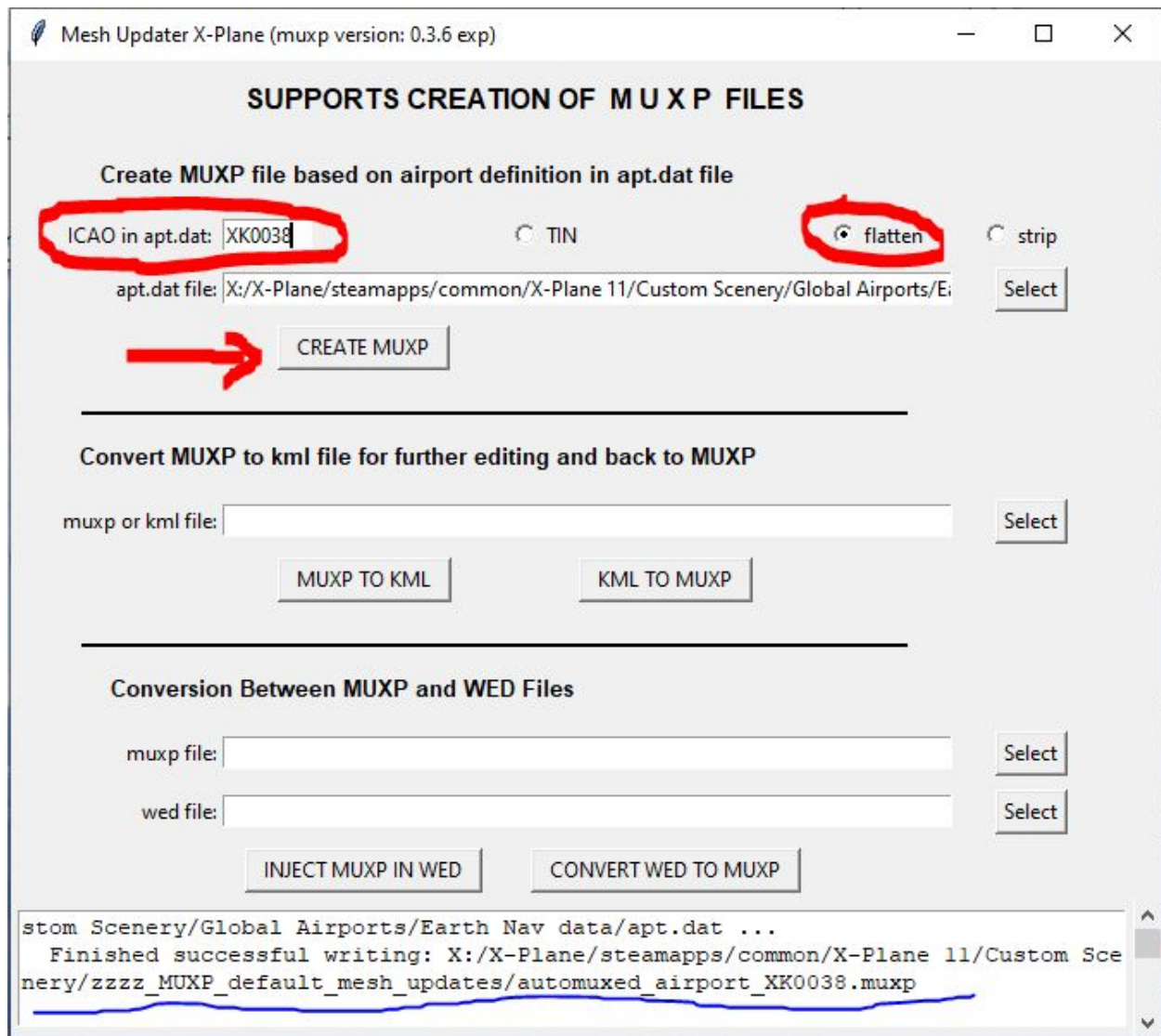


To get it started we use the automatic creation of a MUXP file from the apt.dat. As we adapt a default airport the apt.dat of global airports in the Custom Scenery folder is used. This file is already set as the default path for the apt.dat. As there are thousands of airports in this file, you need to specify this airport by its ID. Usually this is the ICAO Code but for this airport X-Plane used “XK0038” (you can find it when searching in the apt.dat text file for “Lost Bridge Valley” in the same line).

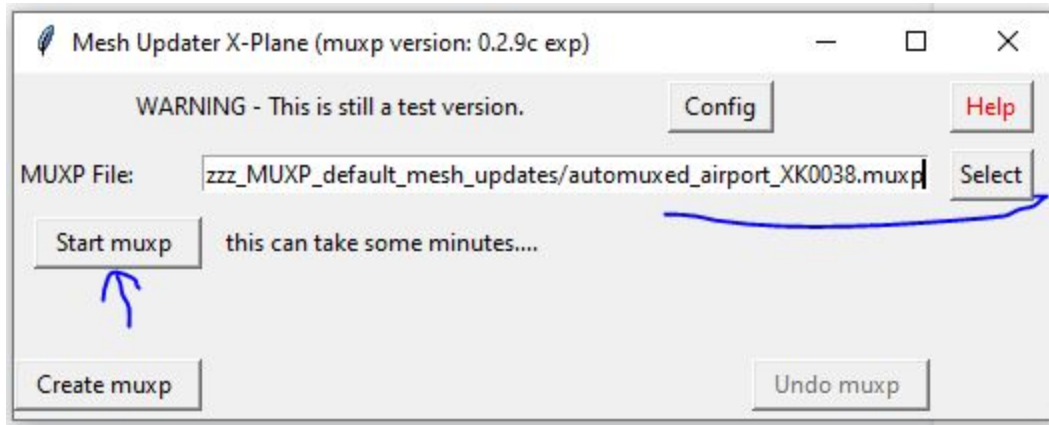
For the first step we use the option “flatten” which sets airport boundaries and runways defined for this airport to the in apt.dat specified elevation of 437 meters.

So after selecting “flatten” click on “CREATE MUXP”. As this apt.dat file is huge it takes some time. When finished you see in the textbox on the bottom of the window a “finished”

message (or errors in case something went wrong). The message includes also the name of the created MUXP file.



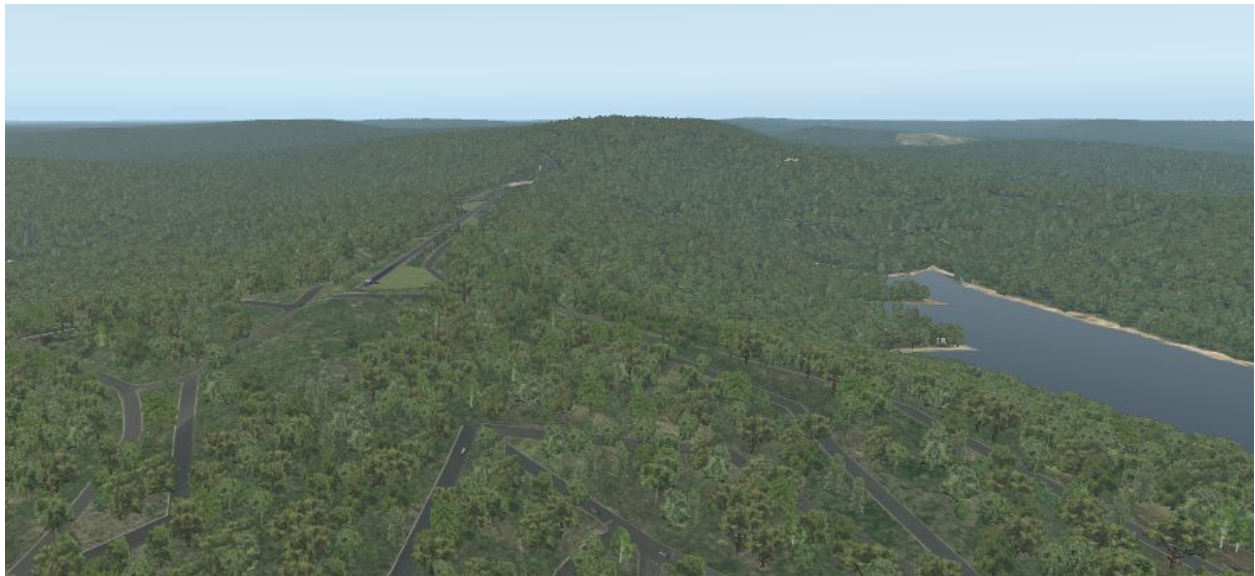
Now you have what you need to run MUXP. Close this "Creation" window, select the newly created MUXP file in the main window and click on "Start muxp".



MUXP is now copying the default X-Plane dsf file to your "MUXP default mesh updates" folder and adapts it as specified in the MUXP file. It also adapts the global airport.dat file in order to remove the flatten flag defined for this airport (proceed in the message window when asked for).

After MUXP finished you could start X-Plane or re-start it in case X-Plane is still running.

i Usually you could use in X-Plane the option to reload scenery from developers option from top menu. However, in case of changes of the global apt.dat you really need to re-start X-Plane to see the change of removing the flatten flag.



The result looks already much better and was also not too much work. However, in reality the runway has quite a curved profile, where our airport and runway is completely flat. So as a next step we are creating a profiled runway with MUXP. Therefore we open again the "Create muxp " window and create a new MUXP file as above but with the "TIN" option

ticked. TIN stands for triangulated irregular network which means that we add more triangles for the runway and airport to better show elevations in the mesh. This option creates a MUXP file where the edges of triangles within the airport boundary are cut so that no edge is longer than 100 meters and it cuts the runway into 50m pieces. Elevations for triangles and runway are taken from four points of the runway defining a spline curve going to these four points. Note, that of course could change the maximum length of edges e.g. to 50 meters and runway pieces to e.g. 20 meters or adding more than the four elevation points when editing the MUXP file in order to get a more accurate mesh, which on the other hand consumes more performance when running X-Plane.

- ⚠ When creating a new MUXP file from apt.dat, the previous MUXP file is overwritten, but the backup file .bak created (which would also be overwritten after confirmation when creating further files).**
- i The new MUXP file does not contain the unflatten command as when creating the second file it was already changed in global apt.dat. So in case you want to share your MUXP files make sure to include this part again (e.g. copy it from the current .bak file and insert it at the bottom of the new file using a text editor).**

As you can see below the result looks better but still not perfect. The runway has a nice profile but the areas close to it as the turning area right of the plane falls abruptly down. Update was not taking this in account. This will work with the option “strip” explained next.



NEW Until MUXP version 0.3.2 the elevation points for the runway profile have only been taken from raster elevation by using - 32768.0 as elevation. This is the value that tells X-Plane to take elevation from raster. In new MUXP versions magic elevation -99999 is used. This value is only used by MUXP. When MUXP sees this elevation it takes the current elevation of the mesh at this point. This can either be a raster elevation (e.g. as in XP-Plane default mesh) or elevation given by vertices (e.g. as done by Ortho4XP).

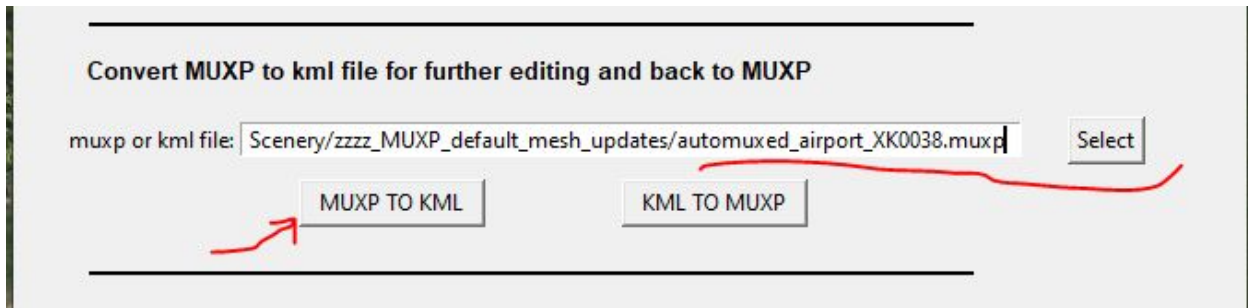
NEW The best way to automatically update this airport is to use the option “strip”. This function automatically updates the elevation of the whole airport boundary according to the spline profile of the runway. Thus it also updates the turning place at end of the runway and parking space along the runway. As this function is mainly suitable for small single runway airports and not large airports it is called “strip”.



Check and Update MUXP in Google Earth

You can convert a MUXP file to a kml file and edit it in Google Earth. Open the Creation Window in MUXP, and select in the middle section your MUXP file and then press “MUXP TO KML”.

This section refers to an older version of MUXP without the automatic muxp creation option “strip” where also raster elevation value -32678 instead of magic elevation number -99999 was used. So all the actions for the airport XK028 are not needed to be done manually any more. However you can still refer to this section how to adapt airports using Google Earth.

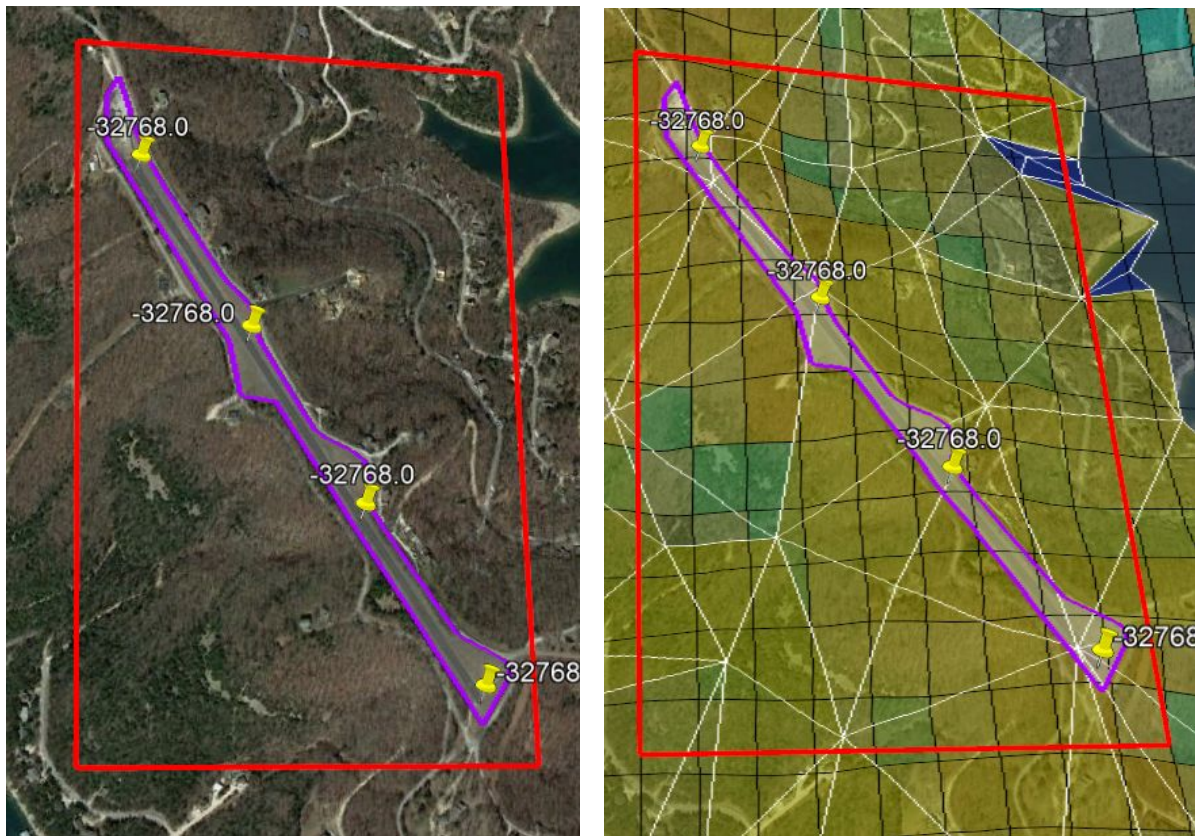


In order to be able to work with Google Earth you need to install the free Google Earth Pro application on your PC (search for it on the Internet, it is free). Then just double click on newly created kml file (automuxed_airport_XK0038.muxp.kml)

Below on the left you see on the satellite image what the MUXP file has for configurations:

- The red rectangle is the area which MUXP will work on. Make sure that all changes you apply to the mesh are within this north aligned rectangle.
- The violet polygon is the airport boundary. All edges of the mesh that are fully or partially inside this polygon will be cut until they reach the maximum allowed length defined by the "limit_edges" command.
- The yellow Pins define the knots of the Spline that will define the profile for the runway.

It is very helpful to also open the mesh export from MUXP (you need to enable the kml export within the configuration menu first, refer to [Configurations settings](#)). On the screenshot on the right the mesh before any update is shown (just double click on the export file "+36-094_dsf_0.kml").



Google Earth allows you to have a 3D view, so you get an understanding of the landscape and you get good elevation values when you go over the image with the mouse, coordinates and elevation displayed on the lower right of the window.

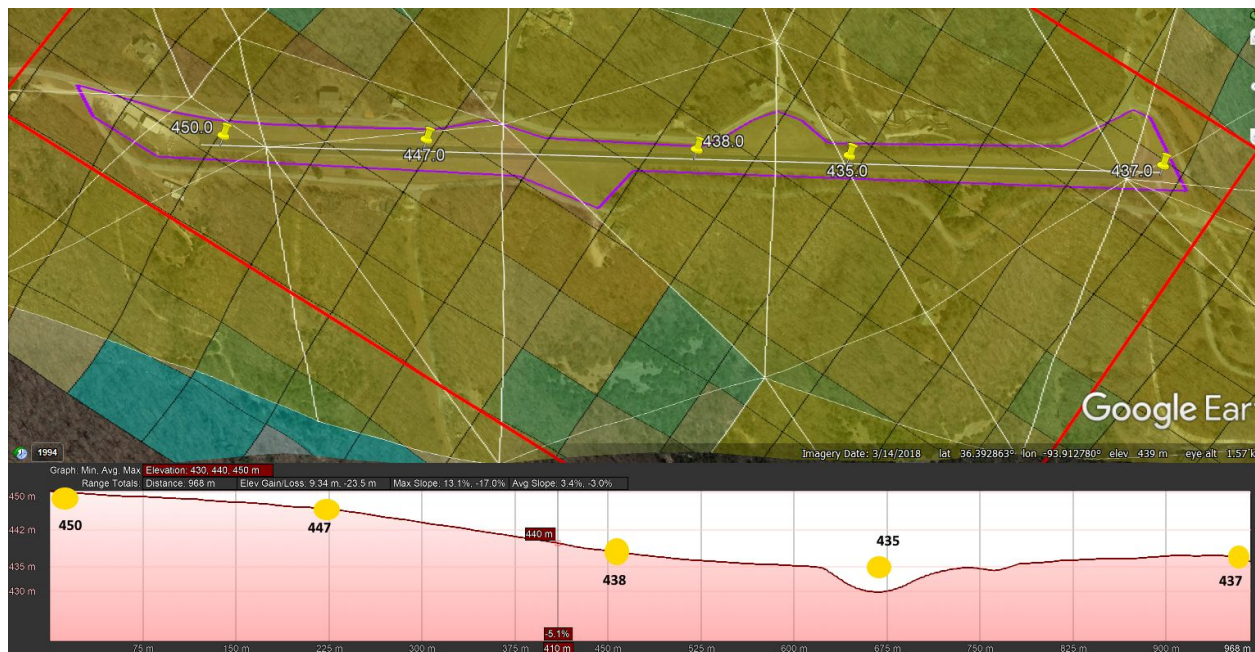
Google Earth can also display you the elevation profile of the runway. Therefore you need

to create a path with one point on each end of the runway by clicking on  in the top menu. The path with its name is shown in the left window (usually "Untitled Path"). Right click on it and select "Show Elevation Profile" and you will see something as below.

Now I used this profile to update the knots of the Spline (yellow PINs). An existing PIN can be moved by right clicking on it and selection of "Properties" from the context menu. You can also create new PINs with the top menu. The elevation is given via the name of the PIN which you also reach via the "Properties".

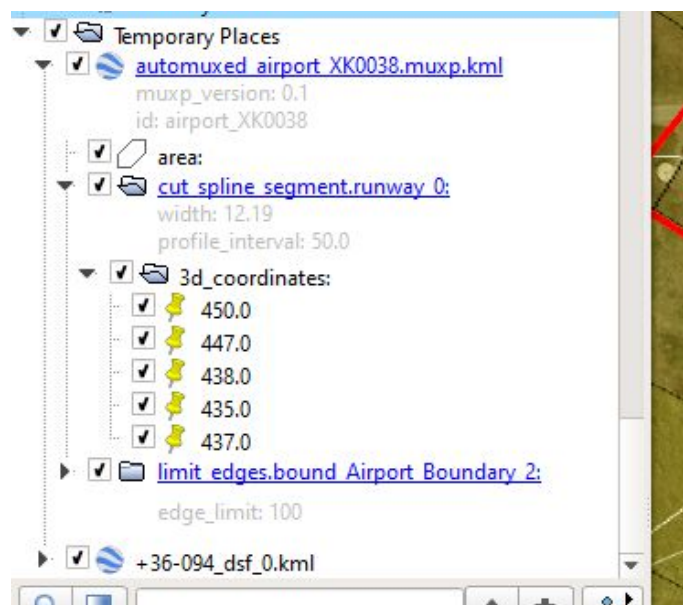
Make sure that all PINs are inside the right folder on the left. They should be inside the "3d_coordinates" folder under the "cut_spline" command folder. Also make sure that additional lines or points you draw that do not belong to MUXP changes like the created path for the profile are outside the MUXP folder on the left.

You can see the spline knots I created in the screenshot below.



Good knots (points) for the spline are the ends of the runways and locations where the inclination changes. In my example the runway is very flat at the beginning, but at the location where it starts to go much faster down I added one knot.

You can also see that the profile is not 100% accurate e.g. the drop will not exist on the real runway, so I estimated (and checked pictures e.g. in Google and Google Street View) that there will be just a flat end, perhaps with some little inclination.



- i** You need to make sure that the pins/points inside the 3d_coordinates_folder have the right order (from one end of the runway step by step to the other end). In addition all points should be nicely aligned on the runway centerline.

Now save your changes by right clicking on the top folder for the muxp file "automuxed_airport_XK0028.muxp.kml" and select "Save Place As.." Take care that you **save it as type kml** (and not kmz which is usually suggested by Google Earth).

In MUXP you can now also directly load the kml file (make sure to select the right file format when searching it in file browser) from the MUXP main window.

Current file converters do currently not really give a lot of help when you have errors in your files. So it could be that the conversion works fine and the MUXP file is not working (e.g. after forgetting ':' after the command name. So for the moment you need to check errors when running MUXP file and learn there what might be wrong in the file you converted.



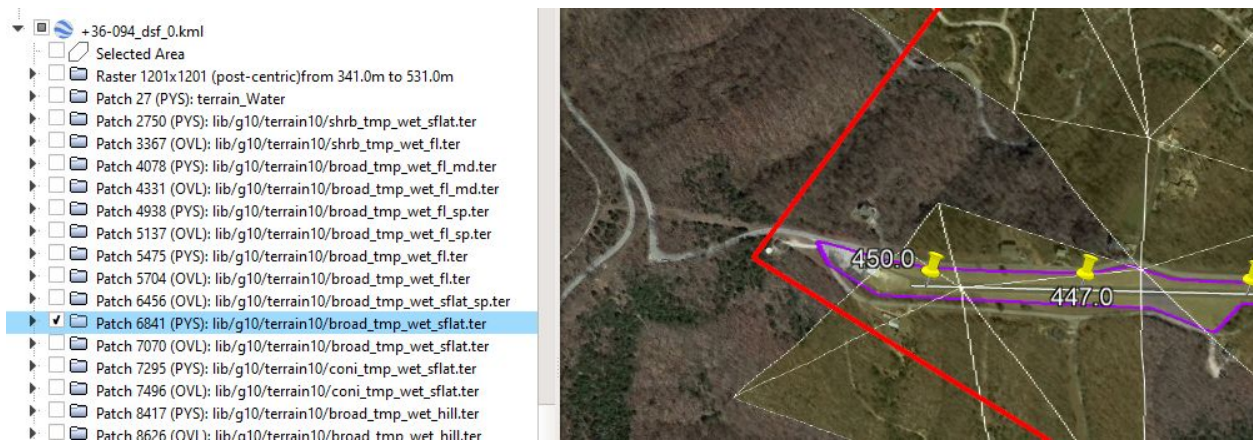
The result looks much better, as you now have a nice profiled runway you can land on. However, it is still not perfect, as now e.g. the parking areas next to the runway drop down, as we only considered to change elevation of the runway

So some more work to be done

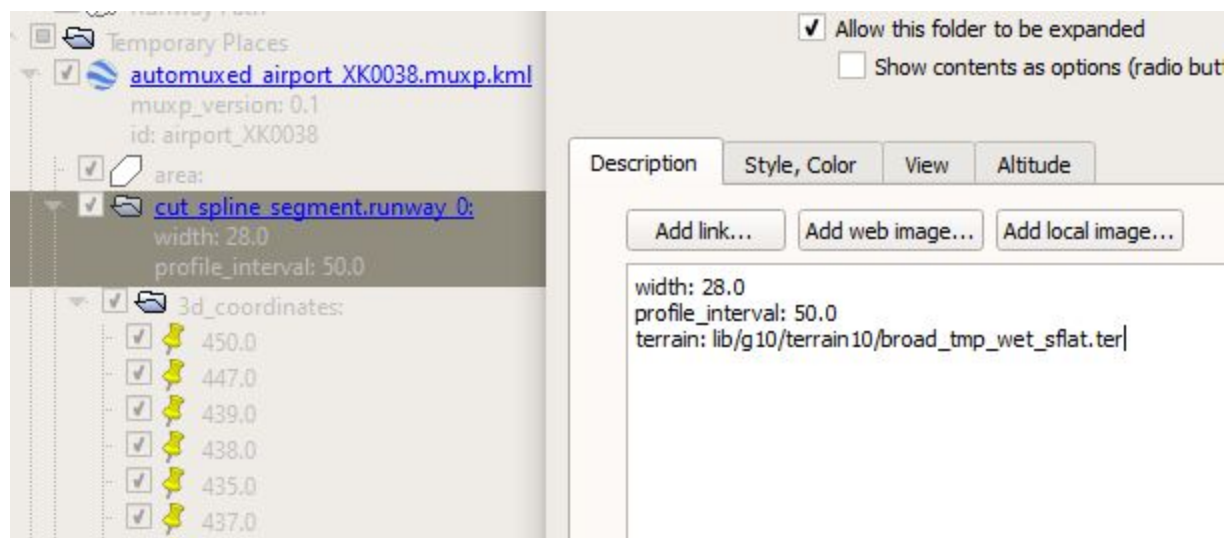
In order to avoid that the terrain is directly cut where the asphalt ends you could enlarge the width of the runway. Click right on the "cut_spline_segment" command, select "Properties" and edit "Description" with runway "width: 28.0" (the blank after ":" is

important) . In order to retrieve the right distance there is a distance measuring function in Google Earth ( from the top menu).

When the width now also includes the shoulders of the runway and not only the asphalt runway you need to define the right terrain. One option is to set the terrain to the same kind as the surrounding terrain of the runway. To find the name for this terrain you could check which terrain was used for that area in the original dsf file. When you untick all the complete +36-094_dsf_0.kml file on the left and just tick the mesh Patches (folders in that file) individually you can find a terrain that matches best. For the time being MUXP only supports usage of physical terrains (check for "(PYS)" for terrain changes.



Now you can set the "terrain : lib/g10/terrain10/broad_tmp_wet_sflat.ter" in Properties.



Another alternative is to set some airport gras as the shoulder of the runway. In the folder X-Plane 11/resources/default scenery/1000 world terrain/terrain10 you find many possible .ter files that could be used. In case of using draped polygons (e.g. orthophotos) by the airport definition you don't need to worry as the terrain is then covered by the polygon.

Now we need to flatten the parking / turn-around area at the south end of the runway. For that we will use a new MUXP command "cut_polygon". To create a new command, right click the top-level folder "airport_XK0038.muxp.kml" and select "Add" then "Folder" from the context menu. Drag and drop it to the right position (e.g. better before cut_spline_segment command).

i It is very important not to change the hierarchy of the folders. It could easily happen that your new command by drag and drop lands inside another command folder. This will cause errors when using the MUXP file. Each command must be a direct subfolder of the main muxp file folder. It could be tricky to get it outside again of a subfolder. You really need to drop it at the very left end of the window between two commands (line is visible no box around a folder).

In kml files the definition of "area:" polygon must always be in the top-level folder, at best at top before all other commands.

i Be cautious with right spelling of commands and don't forget the ":" after each command name and key names like coordinates and area. And between key and value after ":" at least one whitespace " " is required.

i The order of the commands could matter on how the mesh will look like.

For our new command we need to define a polygon that will be cut flat into the mesh.

Therefore choose  from Google Earth top menu and define a polygon for the turnaround area. Don't create too many points (in our case 5 to 7 should be enough) as this would lead to many very small triangles that could create errors in the mesh and consume too much performance.

The elevation for this polygon is defined via the "Properties" of this new command. Google Earth shows for that area an elevation of 437 m which is also the elevation for the last part of the runway. So add to the Properties the line "elevation: 437.0".

The cut_polygon command cuts the mesh triangles and gives the new triangles the same terrain (this is done for physical and overlay triangles). So no definition of terrain needed. There are other commands for changing terrain e.g. when a lake should be cut into the

terrain. As in our case the turnaround area is covered by draped polygon in the airport definition the terrain does not play a role here.



Another option to change "profile_interval" (via "Properties" for the "cut_spline_segment" command) for the runway to 20.0 meters. This means the runway is intersected in smaller pieces and the profile becomes more accurate and smooth, but on the other hand the mesh needs more triangles which means less performance.



So we improved the mesh further. But we have to do the same for the parking areas attached to the middle of the runway.

Other improvements could be to also adapt the mesh for the roads like flatten them or assign spline profiles for them.

For parking areas in the middle of the runway we need another command as there is a slope of runway that needs to be aligned. We could use the “cut_ramp” command. This works as the cut_polygon command but instead of being flat the polygon is laying on a sloped plane. The plane is defined by three 3d_coordinates. So for the parking areas you do the same as for the turnaround area, you only need to add the three 3d_coordinates defining the plane:

- Create in your command folder “cut_ramp” a subfolder called “3d_coordinates:” (right click the command, select “Add” and then “Folder”).
- Create 3 Placemarks (Pins) inside this “3d_coordinates:” folder where you set the name to the required elevation.

As you can see below I defined two points on an orthogonal line crossing the runway at the elevation of 438 meters north of the parking and one on the orthogonal line south of the parking where the runway has an elevation of 434 meters. This means that our parking area has about the same slope like the runway (the slope can not be exact as the runway is curved by a spline and our parking area lies on a plane).



Now you can do exactly the same thing for the parking area on the west side of the runway. Google Earth also supports paste and copy functions for folders. For the copy you need to

change the names and drag and drop polygon points and 3d points to their correct position.



Last but not least we need to handle the taxiway and small hanger on top of the runway. This can be done very well in WED where objects and taxiways are visible and changes can be aligned to them.

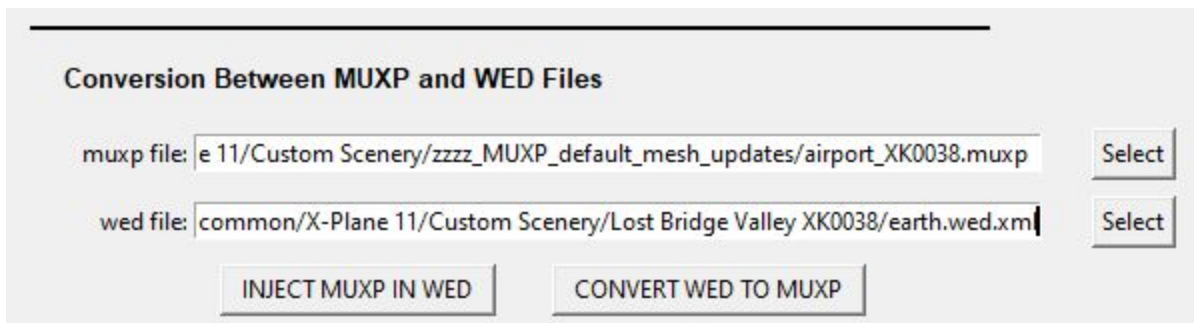
But before using WED we need to have a MUXP file. So in case you did still use the muxp.kml file for converting your mesh with MUXP you need to convert this kml file to a MUXP file by selecting the muxp.kml file and press “KML to MUXP” button in the Creation window of MUXP.

Use WED

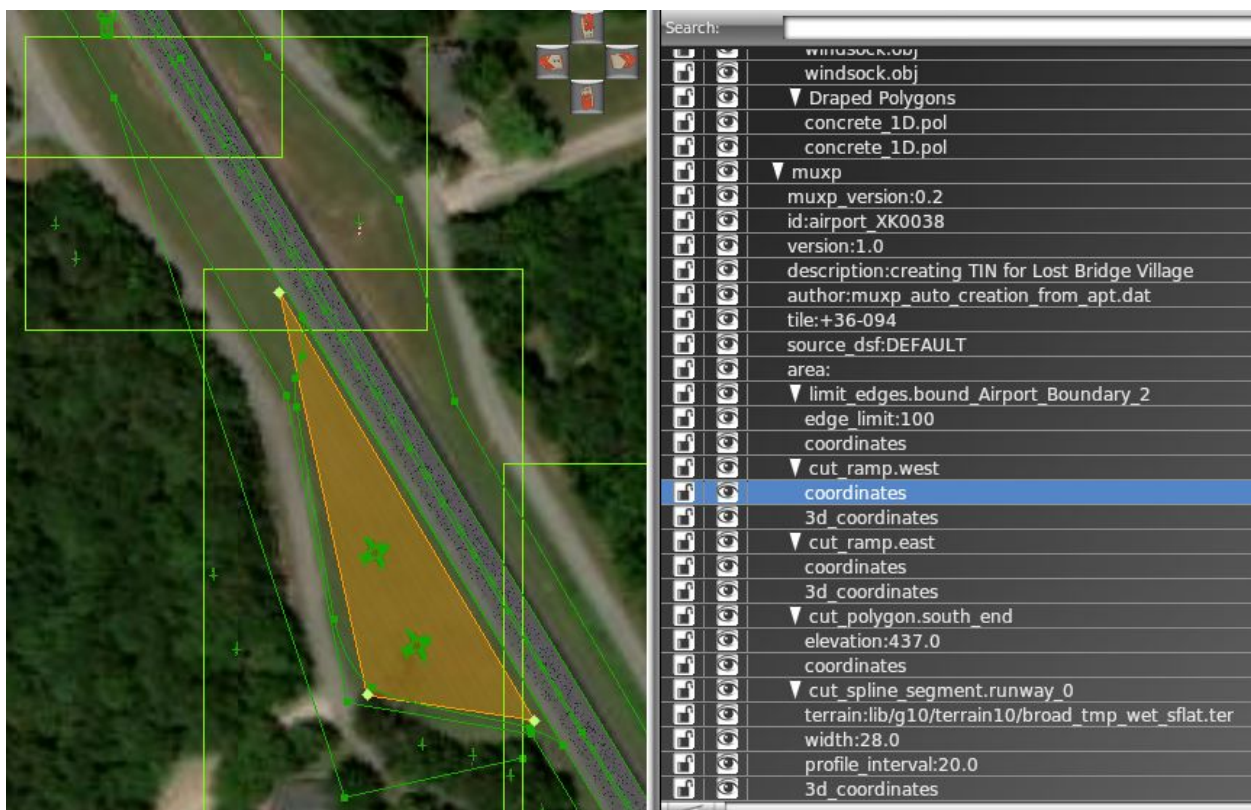
For this section it is expected that you know how to use World Editor X-Plane (WED). If not you could check out several tutorials on the Internet or the manual on:

<https://developer.x-plane.com/manuals/wed/>

To get started you need the “earth.wed.xml” file for our XK0038 airport which you could download from the X-Plane Scenery Gateway. Then you can inject the MUXP file into that WED file and start (or re-start) WED with this updated WED file.



As you can see, the WED file includes a new group “muxp” including all values and commands from the muxp file.



It works as follows:

- All content of a muxp file is within a group called “muxp”
- Values of MUXP file itself like “id” and inside commands like “width” are usually an Object where the name of the object includes the key-value-pair (e.g. “width: 28.0”). The resource information (which object) is usually just empty. As WED has no information on the object “?” is displayed.
- The value “area” is a draped polygon with 4 coordinates building an north-aligned rectangular which includes all changes done by MUXP file.

- Commands like “cut_ramp” are groups where the name is the command name; but in difference to commands in kml the “:” after the command name is not used.
- Coordinates are draped polygons. Also no “:” after coordinates.
- 3d_coordinates are object strings, where the names of the points on the string are representing the elevations. Again “:” are not used in WED (but in kml).

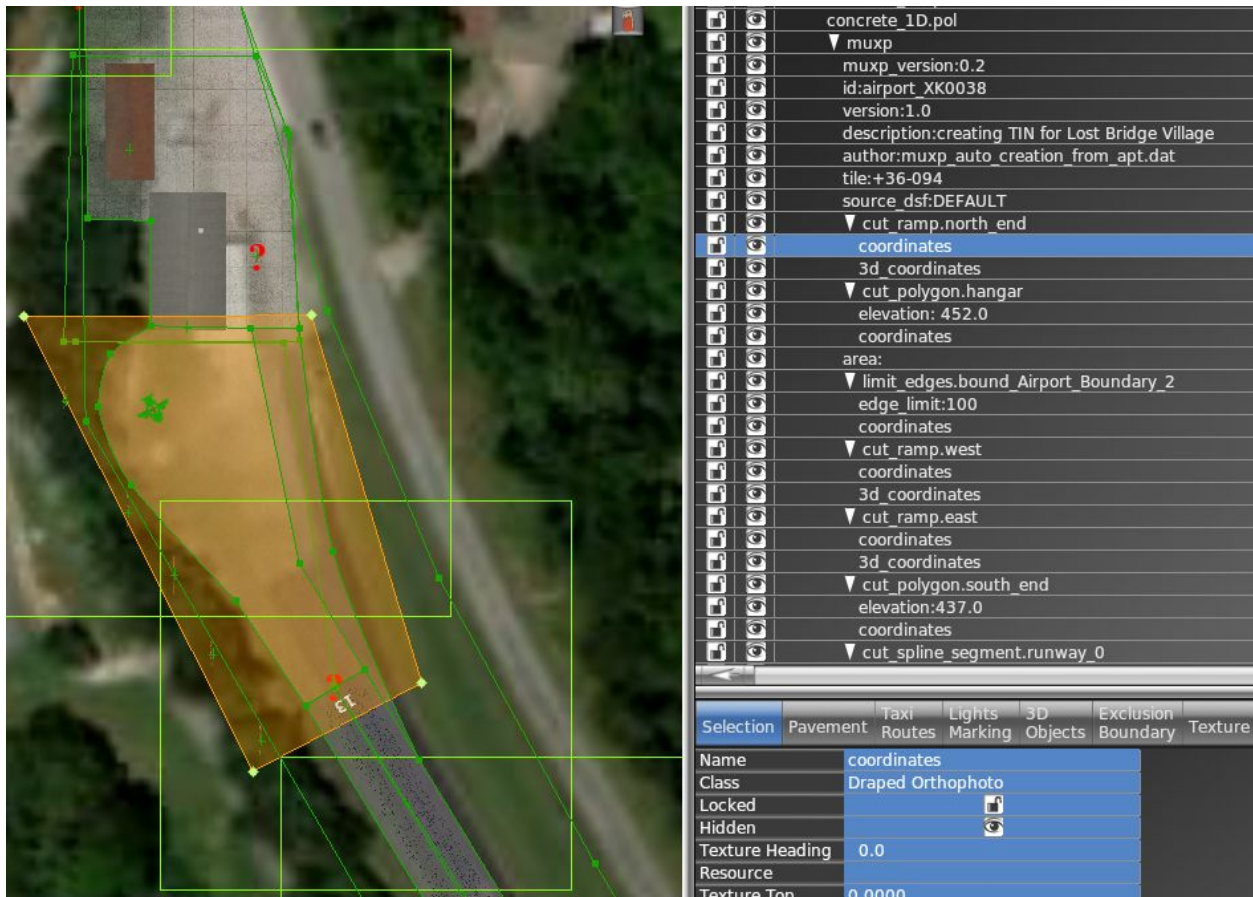
In the screenshot above you can see that the coordinates of the “cut_ramp.west” command are overshooting the draped polygon for the grass parking area that was created by the scenery designer. In WED you can now exactly adapt the elevation changes to the defined scenery. In the case above I aligned the coordinates for the cut_ramp to be inside the draped polygon.

Next we flatten the area of the two small hangars in order they do not sink into an elevated hill or are floating.

- Create a “Draped Polygon” in WED inside the muxp group. Name it “coordinates”.
- Create a new group (top menu Edit → group) when coordinates are selected, so coordinates will be in that group. Name the new group “cut_polygon.hangar”.
- When the cursor is in the new group (on coordinates polygon), select any object (e.g. helicopter from the library) and place it inside the coordinates polygon; remove resource name (so helicopter will vanish and ? is displayed). Set name to “elevation: 452.0”

Do now the same steps as above for a “cut_ramp” command from runway end to the flat polygon of the hangars. Do overlap the polygons for at least 1 meter as there will be triangles created in between. Depending which command comes first it will look different. The overlap will stay ramp in case the cut_polygon command comes first or it will be flattened if the cut_polygon command comes second.

Instead of elevation create a string with three nodes defining the elevations for the sloped plane. (e.g. 2 nodes on hanger line with elevation 452.0 meters and on at runway end with elevation 450.0 meters , where the runway starts at).



Just save your changes in WED (no export needed as we only need earth.wed.xml file). In MUXP you can directly import the WED file for updating the mesh (remember to pre-select the right file ending to see it).

You can also convert the WED file back to a MUXP file again, within MUXP Creation window.

- i Take care of right spelling of keywords and order / hierarchy of the groups within the muxp group.**
- i When making the muxp group not visible (closed eye) in WED you can still use the same WED file for defining the MUXP file and using it for creating the airport. In case of invisible muxp group the information about MUXP will not be exported to the airport definitions.**

There are also many other things that can be adapted, like removing buckles from roads, setting right elevation for buildings etc. Feel free to continue. The good thing about MUXP is that you can then share your mesh update easily with others.

Share MUXP file with others

In case you have created nice MUXP files please share them with others e.g. by uploading them to X-Plane Forum or by distributing them together with your airport scenery. It is much better than sharing dsf files. MUXP files are not only much smaller but this allows that users can apply changes to the type of mesh they want and especially they can have several mesh changes on the same dsf file.

As you have seen, using MUXP files is much easier than creating them, so it would be great if you help others with the effort you already have performed.

In order to further simplify the updating of mesh there is the idea that MUXP can automatically check the Custom Scenery folders for new MUXP or WED files. Especially when using WED files changes could also be distributed via the Scenery Gateway.

Perhaps also Laminar Research would make use out of them when distributing a new release of mesh where all our changes would be considered :-)

MUXP File Structure

MUXP files are defined in a “yaml” format (however makes no use of full yaml functions).

On top of the file the base values define the relevant information for MUXP like the identifier of the MUXP file, its version which area is updated etc. Some of these base values are optional like the author of the MUXP file. After these base values command keys and values define different changes on the mesh.

Example File

The following example updates the airport Ascension Wideawake Field (ICAO: FHAW) on Ascension Island in the South Atlantic Ocean. It is located on the dsf tile coordinates -08 and -15. The airport updated lies within the coordinates specified by the key “area”. This means that MUXP only needs to work on that small area and not the complete dsf tile.

```

muxp_version: 0.1
id: airport_FHAW
version: 1.0
description: TIN for Airport Ascension Wideawake Field
author: schmax
tile: -08-015
area: -7.97825311 -7.96163905 -14.40809251 -14.37793147
source_dsf: DEFAULT

# Now the commands changing the mesh follow

cut_spline_segment.runway:
  width: 45.72
  profile_interval: 50
  terrain: lib/g10/terrain10/crp_wrm_sdry.ter
  3d_coordinates:
    - -7.96341006 -14.40605924 -32768
    - -7.96754049 -14.39780329 -32768
    - -7.97167092 -14.38954733 -32768
    - -7.97580135 -14.38129138 -32768

limit_edges.bound_Airport_Boundary:
  edge_limit: 100
  coordinates:
    - -7.97614821 -14.37843147
    - -7.97286278 -14.38495518
    - -7.9714763 -14.38426445
    - -7.96993533 -14.38735553
    - -7.97015346 -14.39070495
    - -7.96213905 -14.4069493
    - -7.96323215 -14.40756916
    - -7.96887161 -14.40759251
    - -7.97164528 -14.40182888
    - -7.96862502 -14.39747738
    - -7.97775311 -14.37922134
    - -7.97614821 -14.37843147

Unflatten_default_apr: # removes the flatten flag from global apr.dat
  name: FHAW

```

After definition of the base values this MUXP file has three commands. The first creates a spline in order to create a profiled runway. The spline in this example is defined by four 3d coordinates (one at each end of the runway and two in-between). The third value of each coordinate is the elevation in meters. However, in our example we just make use of the elevation defined by the dsf file itself inside it's rasters. The value -32768 is not a real elevation; it is just used in dsf files to indicate that raster elevation shall be used. This is just an example it is up to you to define a spline as you want with elevations you want. But be careful not defining too many spline coordinates as you might end up in an oscillating

runway curve. Best is to just use coordinates for local elevation maximums and minimums on the runway.

As this MUXP file makes use of raster elevation it would not work for e.g. an Ortho4XP file. But if you set correct elevations in meters you could use the file also for default and Ortho4XP mesh. As we have raster elevation here the base key "source_dsf" is set to "DEFAULT" which makes this file to be used by X-Plane default mesh, where raster elevation is always present.

The width defines the width of the runway in meters and the profile interval how long (in meters) at maximum a mesh triangle could be that defines the runway. Smaller values give you a more accurate mesh but will end up in more triangles consuming more performance of the computer when used in X-Plane. The terrain defines the terrain (e.g. gras, sand etc.). Usually it is covered by the runway and you won't see it.

The second command limits the maximum length of an edge of the mesh that lies within a polygon defined by coordinates. In our example coordinates define the boundary of the airport and all edges of the mesh triangles will be cut to be not longer than 100m. By this you assure to get a more accurate elevation mesh for the airport area.

The third command just removes the flatten flag from apt.dat containing all global X-Plane default airports. This is relevant in order to not just see a flat area but the updated mesh. Refer also to the sub-section "Airport Area is Flattened" in Running MUXP Files.

i Following information is relevant for command structure in MUXP files:

- After a command like *cut_spline* you could add *'.'* with some optional information like *"cut_spline.runway"* on the command that helps you to remember what this command is for but which is not needed for MUXP.
- In case you use the same command several times you should add such additional information in order to have a valid yaml file where each key is only allowed once (so you could have command keys like *cut_spline.runway_1* and *cut_spline.runway_2*).
- For commands in MUXP files make sure that all key-/value-pairs belonging to the command have an indent of e.g. 3 whitespaces.
- Between key and value *':'* + at least one whitespace *' '* is required.
- Really use whitespaces and no tabulators (which some editors automatically create).
- Coordinates are given as list elements which start with *'-'*, followed by latitude and longitude and in case of 3d coordinates by elevation in meters (float values like 13.24 are possible). This means that in case of a negative latitude the line starts with *" - -7.9xxx"*.
- You can use *'#'* to start comments. Everything starting at *'#'* is ignored.

Must Base Values

At minimum the following base keys with values are required in order to give MUXP the minimum information required:

- **muxp_version:** This float type is identifying the version of muxp used for this MUXP file. This will allow future versions of muxp to handle older definitions. Current version is 0.1.
- **id:** A string with unique id of the muxp file. This id is also stored in the updated dsf files and thus allows to see what changes have been performed for that updated mesh. Id must not contain spaces.
- **version:** This is the version of the mesh update for that muxp file id. Has to be float but is Independent from muxp_version.
- **tile:** This is the one by one grid tile definition with latitude and longitude as it is used for the dsf files e.g. +50-001
- **area:** This specifies the corner coordinates of the area that will be adapted as 4 float values (lower latitude, higher latitude, lower longitude, higher longitude). This area is extracted, adapted by muxp and inserted again in the dsf file. So make sure that all your changes are within this area.

Optional Base Values

- **source_dsf:** This allows to define for which scenery pack this MUXP file should be used for. It could include several packs separated by whitespace. The preferred can be specified by 'DEFAULT' (for default X-Plane mesh) or by pack=scenery_pack_name. In case the name should not match completely '*' can be used and all after this asterisk must be included in the pack name at any position (e.g. use *pack=*Ortho4XP* in case your MUXP file is working for Ortho4XP dsf files.
- **description:** This string allows you to give some more details and the changes.
- **author:** Allows you to add you as an author for that fiel (type string).
- **elevation_step:** Allows you to set the "resolution" of elevation that will be used for dsf file in float format.

NEW This value is set to 1/51 meter (approx. 0.0196) per default. This means that in an updated mesh the smallest elevation change you could have is about 0.02 meters. Normally you shouldn't care about this value and don't make use of this

option. Reason for this strange value is that the dsf file encodes elevations as a fraction of max integer 65335. Normally your mesh will be connected to meshes that have full meters. In order to match full meters you need a value $1/x$ where x is divisor of 65335. Options are 3, 5, 17, 257 and combinations as $3*17 = 51$. 0.02 meters seems very reasonable also for a very curved runway. When muxp updates a mesh with even higher resolution it automatically takes this as base.

And after this information all needed commands to update the mesh follow.

MUXP Commands

Each command has required parameters and optional parameters. The type of the parameters and some examples of how to make use of the commands is defined below.

Following commands are supported :

cut_polygon:

MUST: coordinates

OPTION: elevation, type [smooth, double_cut], distance

Cuts a polygon defined by coordinates in the mesh. When giving an elevation the polygon will have the flat elevation in the mesh. With "type: smooth" you can avoid steep elevation changes. All vertices around the polygon will smoothly be aligned to elevation which is given by value distance in meters around the polygon. "type = double_cut" will do an additional cut in distance around the polygon. Thus you can easily **define ditches or dams**.

cut_ramp:

MUST: coordinates, 3d_coordinates

OPTION: type [smooth, double_cut], distance

This command is similar to cut_polygon, but the polygon will not be flat. It will be assigned a **sloped plane**, where the plane is defined by the three 3d_coordinates. type and distance have the same function as for cut_polygon.

cut_flat_terrain_in_mesh:

MUST: coordinates, terrain, elevation

OPTION: elevation, type [smooth, double_cut], distance

This command is very similar to cut_polygon. Main difference is that the current terrain is replaced by a specified terrain file (currently only a simple terrain format without s/t

coordinates). By using this command also the triangulation inside the polygon is completely renewed. So this command could also be used to reduce complexity inside the mesh. And it is the perfect command for **creating lakes** with "terrain: terrain_Water".

cut_spline_segment :

MUST: 3d_coordinates, terrain, width, profile_interval

OPTION: type [smooth, double_cut], distance

This command creates a rectangular with length from first to last point of the list of 3d_coordinates and width given in meters (half the width left and half width right from 3d_coordinates). The elevation is defined by a third grade spline defined by the elevation values of 3d_coordinates. Thus at minimum four 3d_coordinates are needed. **This is the typical command to create runway profiles.**

The profile_interval in meters defines, in how many intervals this rectangular will be cut. The smaller the value the more accurate is the profile but the more impact on fps. Usually values between 50 and 20 meters are sufficient. But if you have big changes in the gradient like in Courchevel you might need to go down to e.g. 2 meters. This command re-defines the triangulation by a new terrain file (usually hidden under the runway).

Like for cut_polygon type and distance parameter can be used for smoothing. This is especially valuable to avoid steep climbs from taxiways to the runway.

cut_spline_poly:

MUST: coordinates, 3d_coordinates

OPTION: type [smooth, double_cut], distance

This command creates the spline for a defined polygon shape and not just for a rectangle. All vertices of the polygon get the elevation defined by at least four 3d_coordinates defining the spline.

Like for cut_polygon type and distance parameter can be used to avoid steep cuts.

cut_strip:

MUST: coordinates, 3d_coordinates, terrain, width, profile_interval

OPTION: type [smooth, double_cut], distance

This command combines cut_spline_segment and cut_spline_poly. The rectangular (runway) is cut and assigned spline elevation as for cut_spline_segment. In addition the elevation of the polygon defined by coordinates is also adapted like using cut_spline_poly. This command is very useful for **landing strips** with parking places and turning points next to a profiled runway. When you define the polygon to be the outline of the landing strip

including attached places then they will get the same spline elevation as the runway. In addition the runway will be cut in smaller segments in order to show a smooth profile.

cut_path:

MUST: 3d_coordinates, eleva

OPTION: type [smooth, double_cut], distance

Cuts a path through all 3d_coordinates into the mesh, having the elevation given for each 3d_coordinate and the width given in meters. The elevation from one 3d_coordinate to the next one is adapted linearly. This command is useful to cut the mesh for **taxiways or roads**. When you want to avoid immediate changes of the inclination at a 3d_coordinate you have to define more 3d_coordinates with "softer" changes of elevation.

limit_edges :

MUST: coordinates, edge_limit

OPTION: -

This command limits the length of all mesh triangles that are inside or cut the polygon defined by coordinates. The value edge_limit defines the maximum length in meters that these edges will have after processing this command.

This command is very useful to create a more granular mesh showing more details in case the mesh file uses raster elevation. For default mesh the raster squares are about 90 x 90 meters. In that case it is not useful to have an edge limit that is much lower than 90 meters. As in many areas edges have length of several of hundreds of meters even edge length of about 200 meters can improve mesh quality significantly.

update_raster_elevation :

MUST: coordinates, elevation

OPTION: include_raster_square_criteria [**corner_inside**, center_inside, square_cuts_poly]

Updates the elevation of raster_squares inside the polygon given by coordinates to the elevation assigned in meters. By the optional parameter include_raster_square_criteria you can specify in which case the elevation of a raster square is to be considered to be inside the polygon: if at least one corner of the square is inside polygon (default) or if the center of the square is inside the polygon or if the polygon cuts the square somewhere. Raster squares that are completely inside the polygon are always updated.

Advantage of updating the raster elevation is that you do not touch the mesh itself but just adapt the elevation of all vertices that are inside this square.

△ **This command only works on dsf files having raster elevation included like X-Plane default mesh but will not work e.g. on Ortho4XP mesh.**

update_raster4spline_segment:

MUST: 3d_coordinates, width

OPTION: -

This command is similar to the cut_spline_segment command. But the rectangular shape created by the first and last vertex of 3d_coordinates and the width is used to adapt the raster elevation. So the raster square (if fully inside the rectangular or at least one cordne is inside) gets the elevation that the spline through the 3d_coordinates has at the position of the center of the square. Keep in mind that the raster squares are usually about 90 x 90 meters, so don't create too close 3d coordinates.

This command is very useful to create profiled runways that have no sudden changes in elevation without changing the mesh coordinates and terrain.

△ **This command only works on dsf files having raster elevation included like X-Plane default mesh but will not work e.g. on Ortho4XP mesh.**

update_elevation_in_poly:

MUST: coordinates, elevation

OPTION: -

This is a very simple but efficient command. All vertices inside the polygon defined by coordinates get the given elevation assigned. But mesh structure itself will not be changed. So there is no cut of the polygon shape in the mesh as it is done by "cut_polygon" command. This also means if you define a small polygon where no vertex is inside nothing is changed. Advantage of this command with regard to "update_raster_elevation" is that it works for all kinds of dsf files.

extract_mesh_to_file:

MUST: coordinates

OPTION: name, type [**mercator**, degrees, meters][&][**centric**, non_centric, center4elevation][&][**cut**, inside_polygon]

Extracts mesh inside polygon defined by coordinates to a Wavefront .obj file. Only the physical mesh (without s/t coordinates) is exported without textures. However the export uses two different materials. One for water and one for land (without any terrain information).

When no other filename is specified by parameter name, the filename is "muxp_mesh.obj". The folder for the file will always be the same as the MUXP file.

The parameter type allows you to specify additional options for the export. There are three categories for type as defined below. You can make use of several options by concatenating them with the "&" character. But it must stay as one string with no blanks.

Projection of coordinates	mercator (default)	Coordinates are converted from degrees to Mercator coordinates. Advantage is that the angle between edges will be exactly the same in exported mesh as later in X-Plane.
	degrees	Especially useful if used with the option non_centric. Then you have exactly the coordinates used in X-Plane, MUXP files, WED etc. So could be helpful for finding errors etc.
	meters	Should only be used with option centric (default). Then you see for each vertex x, y, z distance to the center and you can easily see how many meters two vertices are apart from each other. In addition, it is very useful if you apply an orthophoto as texture for better editing the mesh directly seeing where each vertex is sitting.
Centricity	centric (default)	You should use centric export in order to get small values for coordinates and thus avoid inaccuracy by rounding. Alle vertices are exported as difference in latitude and longitude to the center of the exported area.
	center4elevation	This option can be used in addition to centric. In that case also the elevation is relative to the center. This is useful in case you adapt mesh in a high mountainous area.
	non_centric	Coordinates are exported as they are. This is useful e.g. for error checking in case you use option degrees.
Extraction	cut (default)	Using this option the exported area is exactly cut at the edges of the polygon. So the shape of the exported area is equal to the shape of the polygon.
	inside_polygon	Using this option only mesh triangles are exported that are lying inside the polygon. So the shape of the exported structure is an irregular shape that fully lies inside the polygon. Advantage is that no cutting is used and thus all exported triangles are unchanged as in original mesh. Could be used in case the cutting causes errors.

Example: meters¢er4elevation&inside_polygon would extract the mesh with coordinates in meters relative to the center point of the extracted mesh, where also the z-coordinate (elevation) would be centered. Extraction would only happen for mesh edges that are fully inside the polygon.

Adapting mesh with a 3d computer graphics program such as Blender is very useful. Adapted mesh can be imported again with the following import command. When doing this you should consider the following:

- When exporting mesh to a file you usually don't update the mesh, so you could use the "exit_without_update" command (see below) to directly terminate muxp processing after extraction.
- Mesh exported with MUXP is done in x, y, z coordinates where x is longitude, y latitude and z elevation. Blender uses the same orientation but assumes y coordinate up when importing .obj files. So when importing to Blender select in the import options (comes up after selecting "import .obj") **y forward an z up**.
- In Blender you usually need to set a larger maximum clipping level such as 50,000 meters as the mesh of an airport is usually several thousand meters long.
- If you export centric don't touch the **CENTER object**. This stores the information on the center and must not be changed in order to get the mesh back to the right position.
- You might consider to texture the mesh. At minimal the water with blue and land e.g. green color. Even better option is to texture with an **ortho image**. But keep in mind that re-import only works on the plain physical mesh with the two materials for water and land.
- When texturing with ortho images the best option is to cut out a rectangular north-south/east-west mesh, centered in meters using the same coordinates as the corners of the ortho image. Then (in blender) use "project from view". However, usually you still need to scale the ortho image on the mesh. Works best when you fixate one coordinate to one point of the mesh where you have the exact location and then use the scaling options for the texture applied.

insert_mesh_from_file:

MUST: -

OPTION: coordinates, name, terrain, type [**mercator**, degrees, meters][&][**cut_obj_otuline**, exact_match, same_cut, fill_gap, same_outline_inside_polygon]

By this command you could import mesh that is defined as in Wavefront .obj file format to the dsf file. The existing mesh at the area imported is then fully replaced by the imported mesh. So this option allows you to inject mesh from another dsf file that was extracted by the command above and/or mesh that was adapted with 3d graphic software like Blender.

However, there is one big limitation, that **only physical mesh is imported of one single terrain type** (without s/t coordinates) **for land** (material needs to be named "solid_default_terrain" **and one for water** (material needs to be named "terrain_Water"). This is not an issue as long as you edit an airport area where you could limit the terrain to the airport gras or in case the adapted area is covered by airport/scenery definition by a draped polygon.

The **X-Plane terrain** that is used for the land is defined by option "terrain" (in case this option is not used "terrain: lib/g10/terrain10/apt_tmp_rain.ter" is used as default).

When no other **filename** is specified by parameter name, the filename is "muxp_mesh.obj". The folder for the file will always be the same as the MUXP file.

Depending on the type of insertion (refer to the table below) the coordinates will either be taken from the .obj file or from the parameter coordinates that define a polygon for insertion. In case the coordinates in the .obj file are all relative to a **center** then the .obj file needs an object called "CENTER" (usually defined by export command above). The mesh itself always needs to be in an object called "MESH".

The parameter type allows you to specify additional options for the import. There are two categories for type as defined below. You can make use of several options by concatenating them with the "&" character. But it must stay as one string with no blanks.

Projection of coordinates	mercator (default)	Coordinates of vertices in the .obj file are in Mercator projection.
	degrees	Coordinates are in degrees.
	meters	Coordinates are in meters (usually relative to a center)
Insertion	cut_obj_outline (default)	This option retrieves the outline from the mesh to be inserted from the .obj file and cuts the mesh of the dsf file accordingly.
	exact_match	This option also retrieves the outline from the mesh of the .obj file and removes all triangles in the mesh of the dsf file that are inside that outline. In order to insert without missing mesh parts the edges of the mesh outline needs to exist exactly in the mes of the dsf file. Warning: Only works for dsf files you know exactly e.g. because it was the mesh used for extraction and outer shape of extraction was not changed.
	same_cut	This option cuts the mesh with the polygon defined by coordinates and extracts all mesh triangles inside the polygon. Then the mesh from the .obj file is inserted. It

		expects that the cut produces exactly the same vertices in the dsf file as the outline of the inserted mesh has. Warning: Only works for dsf files you know exactly e.g. because it was the mesh used for extraction and outer shape of extraction was not changed.
	fill_gap	This option removes all mesh triangles that are inside the polygon (no cut), inserts the mesh from .obj file and fills the gap between cut and insertion with triangulated mesh of terrain type specified. This type is usually used for inserting mesh that was extracted by using type "inside polygon" in order to avoid cuts. Warning: As the gap is only filled by solid terrain, water areas e.g. of rivers might miss in this area.
	same_outline_inside_polygon	This option is similar to fill_gap but requires that the outline of the inserted mesh is identical to the mesh of the dsf file. Warning: This command will probably decommissioned.

Example: degrees&exact_match will import .obj file with coordinates in degrees and the edges of the outer line of inserted polygon still exactly match the existing mesh in the dsf file where the mesh is inserted.

exit_without_update:

MUST: -

OPTION: -

Exits MUXP without changing dsf, especially used when you just export to an .obj file and no changes to the mesh are applied.

update_network_levels:

MUST: coordinates, road_coords_dropped

OPTION: -

With this command you can change elevation levels of X-Plane networks, especially roads. The levels are used to decide which road crosses which. If they have both the same level (usually 0) you have crossroads. On level 1 roads will be on some kind of ramp and on level 2 one road will cross the other over a bridge. You could go even higher to get bridges over bridges. This command is useful as with elevation changes of the mesh you could directly adapt the roads.

In X-Plane networks (roads) are defined as a chain of points. In order to select the chain you would like to adapt you need to identify this chain with at least two points that are inside

the polygon defined by coordinates. So imagine you have roads crossing from north to south and from east to west. By defining a narrow polygon from north to south you can select the chain for the north-south road and a narrow polygon from east to west would select the other road.

With the chain (road) selected by the coordinates their level can now be adapted. The level for each point of the chain (road) is taken from its closest road_coords_dropped coordinate (first two values represent coordinates by latitude and longitude). The 3rd value of road_coords_dropped is the level of the road that will be assigned.

The current version of MUXP does not export the networks e.g. to kml for viewing and understanding how the network looks like. However, you could use X-Plane Overlay editor which clearly shows networks chains.

unflatten_default_apr:

MUST: name

OPTION: -

Some of the default airports have in the apr.dat for global airports (X-Plane default airports) a flatten flag set. This forces X-Plane to create a huge flat area around the airport and you would not see any of your mesh updates in that area. This flag can be removed manually from the file by the user or MUXP could adapt this apr.dat file (and create a backup file). Therefore MUXP needs to know the identifier (usually ICAO code) for this airport used in the apr.dat file.

This removal of the flag currently only works for the global default airports. For additional airports it is expected that this is done by the scenery designer and for older airports the user has to remove the flag manually from the file.

calculate_vertex_normals:

MUST: coordinates

OPTION: -

Vertex normals are required for correct shading of the scenery. Changing the mesh usually also requires updating the vertex normals (e.g. to assure that a flat part that was lying in the sun moved by creating a slope outside visibility of the sun and needs to be shaded darker).

For mesh with raster elevation the vertex normals are calculated from raster and updates are not required. So e.g. when updating default mesh no further update of vertex normals is required. But changing mesh without raster like Ortho4XP you might see strange shadings in X-Plane and updating of vertex normals is recommended.

All vertices inside the polygon defined by coordinates get new vertex normals. The normals are calculated by all attached face normals.

This kind of calculating vertex normals will smooth changes of inclination. However, if you don't want to have a smoothing e.g. sudden drop from flat going down for a ditch then the upper flat part shall not receive shading to try to smooth this drop. To avoid this you need some kind of double cut at these edges (called "bevel" in Blender).

The parameters do have the following values:

i Parameters of a command are intended by usually 3 blanks (' '). There must always be ':' and at least one blank (' ') between parameter and value(s). List values start in the next line separated by newlines and always have '-' and at minimum one blank (' ') at the beginning of the line. Refer to examples below for better understanding.

coordinates:	- float float - float float - ...	List of coordinates starting in the next line as latitude and longitude of the coordinate (e.g. - 48.86 2.24 for Paris in France) Important: coordinates usually define a polygon. In that case the first and last coordinate tuple needs to be exactly the same in order to create a really closed polygon.
elevation:	float	Elevation in meters (e.g. 12.5 for 12.5 meters) Special values: -32768 is a special value used by X-Plane in order to retrieve elevation at this coordinate from elevation raster (only works for dsf file with raster elevation present). -99999 will be filled in by muxp getting current elevation at this point from the mesh
3d coordinates:	- float float float - float float float -	The first two values are the same as coordinates and the third is the elevation at that coordinates in meters (e.g. - 48.86 2.24 36.2 for elevation 36.2 meters in Paris)
terrain:	string	String giving reference to the terrain file in X-Plane (e.g. lib/g10/terrain10/apt_vhot_rain.ter) None as value (currently only for cut_flat_terrain_in_mesh) will not assign any terrain and in X-Plane you will have a hole in the mesh. You could use this option e.g. for creating entry points of

		tunnels (the tunnel itself has to be a scenery object to be created).
name:	string	Can be any string giving further information, especially used for file_names for insert/extract mesh from/to file (e.g. airport.obj). Default filename: muxp_mesh.obj
type:	string	Gives additional information on a command such as smooth or double_cut for cut commands or details on .obj import/export. Check commands for details.
distance:	float	Distance in meters around a polygon in which elevation will be smoothed or in which a double_cut around the polygon will be performed (default 20.0 meters).
edge_limit:	int	Maximum length in meters allowed for an edge of the mesh (e.g. 100 will make sure that edges inside polygon are not longer than 100 meters)
profile_interval:	float	Defines the maximum length in meters for a segment interval in which the rectangle (runway) is cut by the cut_spline_segment or cut_strip command.
include_raster_square_criteria:	string	This value will be assigned one of the following strings: "corner_inside, center_inside, square_cuts_poly" in order to decide if a raster square is to be considered to be inside a polygon and elevation is to be adapted by updating the raster_elevation.
road_corords_dropped:	- float float int - float float int ...	The first two values define coordinates by latitude and longitude. The third integer value defines the network level which will be assigned to closest network points.

The following version of this document will describe the usage of commands in more detail. For the moment you can check out tips and tricks below and you could try to figure out how commands work by checking the MUXP file examples.

Undo Mesh Updates and/or Uninstall MUXP

Undo function is currently not supported. This means you would need to undo changes manually, which is not too complicated. For default mesh you just need to remove the updated dsf file from you "MUXP default mesh updates" folder inside X-Plane Custom Scenery folder. You just need to go down to the dsf file which you don't want any more e.g. to folder *"X-Plane 11/Custom Scenery/zzzz MUXP default mesh updates/Earth nav data/-10-20"* and there delete e.g. *"-08-15.dsf"*.

In case of updated Custom Scenery just go to the according *"Earth nav data"* subfolder inside the Custom Scenery folder and replace there the current dsf file with the ending *".original"*. E.g. go to folder *"X-Plane 11/Custom Scenery/zOrtho4XP_-08-015/Earth nav data/-10-20"* and delete e.g. *"-08-15.dsf"* and move *"-08-015.dsf.muxp.original"* to *"-08-15.dsf"*.

In addition you could also remove all according backup files like: *"-08-015.dsf.muxp.backup"*.

In case for an updated airport a flatten flag was removed you can reset it again by just setting the comment line to a normal line in the apt.dat for global airports. E.g. open *"X-Plane 11\Custom Scenery\Global Airports\Earth nav data\apt.dat"* and search for MUXP (in case you have several flatten flags for different airports removed,, make sure you are in the right airport section). Then replace *"# 1302 flatten 1 # removed flattening by MUXP"* with just *"1302 flatten 1"*.

MUXP does not change any registry keys or X-Plane settings. So if you would like to uninstall MUXP you could just delete the muxp programs you don't need any more including muxp.config and muxp.log. You might also consider to delete the following backup files:

- *X-Plane 11\Custom Scenery\Global Airports\Earth nav data\apt.dat.beforeMUXP*
- *X-Plane 11\Custom Scenery\scenery_packs.ini.beforeMUXP*
- *X-Plane 11\Custom Scenery\scenery_packs.ini.backupMUXP*

⚠ In case of uninstall you could consider to just delete the whole folder where the MUXP program resides. But be aware if MUXP lives inside your "MUXP default mesh updates" folder you also delete all updated default X-Plane mesh. This can also be wanted in case you would like to return to the X-Plane original mesh.

It is planned that a future version of MUXP will allow to undo individual and also all performed changes automatically.

Basic Knowledge on DSF Files

Understanding how dsf files are defined and work helps a lot when you want to update them. Most important is to know that there are two types of elevation data: elevation by raster (which is used by default X-Plane mesh) and elevation by vertex (which is e.g. used by Ortho4XP). So raster-commands would e.g. not work for Ortho4XP meshes.

In future this section should give more information how the dsf files work.

Outlook & Feedback

MUXP is still under development. Known errors need to be corrected but also some more functions are planned. If you want to see the details you can refer to the issues I'm currently working on: <https://github.com/nofaceinbook/muxp/issues> and on new functions / updates: <https://github.com/nofaceinbook/muxp/projects/1>

If you have any issues or any suggestions I would be happy to get your feedback, best via github issues or via the X-Plane.org forum.

Credits

First of all I would like to thank all who tested and used my previous tool **bflat** (<https://forums.x-plane.org/index.php?/files/file/52153-better-flattening-for-x-plane-airport-s/>) for better flattening of the X-Plane mesh. This motivated me a lot to create this advanced tool MUXP.

Also discussions and ideas of Matthias Seewald (<https://maps2xplane.com/>) did influence this tool. Especially his own tool and ideas for a more general mesh procedure <https://maps2xplane.com/2019/08/22/editing-x-plane-meshes-implications-and-limitations/> inspired me.

Very important for MUXP is also X-Plane.org user nicotum-xp. He did not only test MUXP but also contributed to MUXP directly. Especially to mention his programmed conversion between MUXP and WED files (https://forums.x-plane.org/index.php?/forums/topic/224944-muxp_conv-creating-muxp-files-with-wed/) which is now part of MUXP directly.



What also motivated me is that pre-versions of MUXP have also been used in two commercial sceneries. It was used to create profiled runways and adapt elevations around the airport for https://store.x-plane.org/EGBB--Birmingham-Airport_p_1312.html and for Gedansk (EPGD) in https://store.x-plane.org/Polish-Airports-volume-1-v3_p_1256.html.

Special thanks go to moter for creating the MUXP logo and giving me much input on the import and export functions of .obj files.

Not to forget the huge freeware creators community for X-Plane which makes this simulator so much better. MUXP is my contribution.